ENGINEERING SYSTEMS / USER GUIDES

ArquitecturaRepo

The Complete User Manual

VERSION 1.4 / ENGLISH

Eight full chapters: discovery, installation, adoption, operation, changes, recovery, acceptance and a worked product example.

MATÍAS ACUÑA · NOVEMBER 2026

Source-backed documentation for the public ArquitecturaRepo framework.

Contents

00 — Before You Begin **5**
In this chapter . . . 5
1. What you install—and what you do not . . . 5
2. Who makes each decision . . . 6
3. What must remain accessible . . . 7
4. How to use this manual . . . 8
5. The product decisions only you can make . . . 9
6. Scope, limitations, and versions . . . 9
7. One complete example, from idea to acceptance . . . 10
8. Common decisions and their owners . . . 10
9. A one-page project starter brief . . . 11
10. How to know whether it is helping . . . 11
Sources and applicability . . . 11

01 — Installation and Configuration **12**
In this chapter . . . 12
1. Environment and accounts . . . 12
2. The starter, the fork, and the application . . . 13
3. Verify the location before changing anything . . . 14
4. Select a proportionate profile . . . 14
5. Generate the control layer . . . 15
6. Validate without manufacturing acceptance . . . 16
7. The next action is discovery, not coding . . . 16
8. Before attempting installation again . . . 16
9. From scaffold to genuine adoption . . . 17
10. Connections, access, and privacy . . . 17
11. Safe troubleshooting . . . 17
Sources and applicability . . . 18

02 — The Planner and Project Adoption **19**
In this chapter . . . 19
1. A conversation is not the project . . . 19
2. Prepare the working space . . . 20
3. Discovery: ask the right questions . . . 20
4. Technical baseline and delivery plan . . . 21
5. Adoption: connect the starter to reality . . . 22
6. The real examination: a fresh context . . . 22
7. The first authorized engineering campaign . . . 23
8. An example first Planner session . . . 23
9. A baseline the CLIENT can actually confirm . . . 23
10. An example of a first campaign . . . 24
Sources and applicability . . . 25

03 — Daily Operation **26**
In this chapter . . . 26
1. Start a session without starting over . . . 26
2. Turn a wish into a verifiable capability . . . 27
3. What I3 does when work becomes difficult . . . 27

4. Reserved authorizations and external actions . . . 28
5. Read an executor report critically . . . 28
6. Git and continuity . . . 29
7. The five-question daily closure . . . 29
8. A working day without depending on chat memory . . . 29
9. An example of a bounded dispatch . . . 30
10. Interpret controller states . . . 30
11. Process the final report . . . 31
12. Leave a recoverable session . . . 31
Sources and applicability . . . 31

04 — Requesting Changes 33
In this chapter . . . 33
1. Start with the outcome, not the code . . . 33
2. A compact change-request brief . . . 33
3. Which questions belong to you? . . . 34
4. Change your mind without destroying history . . . 35
5. Ask for quality without prescribing a stack . . . 35
6. Prioritize instead of opening ten Goals . . . 36
7. When the Planner needs a real decision . . . 36
8. Five well-formulated requirements . . . 37
9. Product change versus implementation defect . . . 37
10. What to say when it works but is not useful . . . 38
11. A short priority-setting script . . . 38
Sources and applicability . . . 39

05 — Errors, Incidents and Recovery 40
In this chapter . . . 40
1. Preserve evidence first . . . 40
2. Environment and account failures . . . 41
3. When the control layer fails validation . . . 41
4. Git is not an “undo problems” button . . . 42
5. Repeated failures require a causal change . . . 42
6. Security, privacy, and third-party rights . . . 43
7. Exit an incident with evidence . . . 43
8. Three complete recovery examples . . . 43
9. Decision table for sensitive operations . . . 44
10. What successful validation does not prove . . . 45
11. The starter’s privacy and feedback controls . . . 45
Sources and applicability . . . 46

06 — Closing Capabilities and Acceptance 47
In this chapter . . . 47
1. A capability is a product result . . . 47
2. Five components of closure . . . 47
3. The exact candidate is the review unit . . . 48
4. Missing evidence does not require an artificial campaign . . . 49
5. Accept, resume, or change objective . . . 49
6. Periodic review and the stopping rule . . . 50
7. An auditable closure record . . . 50
8. Rejection example: correct without restarting . . . 51
9. Usable product versus green framework . . . 51
10. When to end a session or stop work . . . 52

Sources and applicability 52

07 — Worked Example: A Club Management Application 53
In this chapter 53
1. The example product: a club with concrete needs 53
2. Ask questions that change the product 54
3. Install without confusing a starter with an application 54
4. First capability: member management 55
5. From Planner to execution: a complete request 55
6. What to test before acceptance 56
7. Second capability: dues, without expanding scope 56
8. Evaluate the whole example 57
9. The first cycle with fictional data 57
10. Record a finding and route it correctly 57
11. The second cycle depends on the first 58
12. Final exercise: recover the project in a new context 58
Sources and applicability 58

00 — Before You Begin

ArquitecturaRepo · Complete User Manual · v1.4 edition · English

> This chapter explains the product, its limits, and who decides what. It accompanies the published v1.4.0 release. It is documentation, not evidence that a particular application has been built, secured, or accepted. Commands and compatibility boundaries are checked against the current first-user guide and release notes.

In this chapter

1. What you install—and what you do not
2. Who makes each decision
3. What must remain accessible
4. How to use this manual
5. The product decisions only you can make
6. Scope, limitations, and versions
7. One complete example, from idea to acceptance
8. Common decisions and their owners
9. A one-page project starter brief
10. How to know whether the framework is helping

1. What you install—and what you do not

ArquitecturaRepo is a portable engineering-control starter. It provides public operating protocols, reusable templates, installation profiles, machine-checkable contracts, Python validation and recovery utilities, and an optional local campaign controller. Together these mechanisms organize software development when work passes between people, programming agents, reviewers, and new conversations. They do not constitute a finished product-building service. Installing the starter does not mean an agent can take a broad request and deliver a validated application without further decisions, work, and evidence.

The published version is identified by a specific Git tag and release assets—not by a version string alone. For v1.4.0, the distribution includes a portable ZIP, its checksum, and a manifest. Verify the downloaded bytes before installation. The continuously updated `main` branch can contain documentation fixes that are not part of the already published ZIP. The framework's release is evidence about its own source and tested mechanisms; it says nothing about whether an unrelated application built with it works.

An application needs its own requirements, technical choices, real execution, tests, risk controls, and final acceptance. The repository contains synthetic adoption examples and a physical reference application to exercise selected framework behavior. Such fixtures support the framework's claims but are not a substitute for testing the project you actually want to ship. A capability that has not passed its applicable real-world gate should remain UNKNOWN, HOLD, or explicitly unsupported—not be described as completed because a script or agent predicted success.

The practical benefit is continuity. A new session should be able to inspect durable

repository evidence and reconstruct: what is being built, which decisions still apply, which source version and technical state are current, what work is authorized, what was actually tested, and what remains uncertain. Chats can end, accounts or models can change, and an agent may lose its context. Your project should not require someone to remember the entire conversation in order to continue safely.

The manual uses GitHub, Windows, Python, and conversational coding agents in examples because they are familiar starting points. They are not universal dependencies. A project may use another operating system, another programming stack, different tools, or only human workers. External models, cloud services, credentials, and paid provider calls are optional and require separate authorization. Installing ArquitecturaRepo does not grant those permissions.

It is especially important to distinguish three statements:

- The starter is installable and its portable mechanisms pass their own tests.
- Your application has adopted the starter with valid, current authority and state.
- Your application satisfies its functional, security, reliability, and human-experience requirements.

They are separate claims. Passing the first does not imply the second or third.

2. Who makes each decision

The framework separates decisions by responsibility, not necessarily by the number of people, AI models, chats, or programs involved. A small project can use one human product owner, one planning role, and one executor. A dedicated Master Owner becomes valuable when multiple products, shared resources, or cross-project policies require separate oversight. Merely creating a chat named “Master Owner” does not establish such a need.

Role	Owns	Should not routinely do
CLIENT / product owner	Problem, intended users, priorities, product behavior, subjective quality, spending limits, publication, and other reserved authorizations.	Resolve merge conflicts, hand-edit governance records, design tests, or repair broken currentness.
I1 · Master Owner (when needed)	Shared operating system, inter-project coordination, systemic quality, capacity, and ownership boundaries.	Replace the project Planner or control every small task.
I2 · Planner	Product interpretation, technical strategy, capability boundaries, architecture, charter acceptance criteria, and strategic acceptance.	Become the debugging loop or rewrite the campaign for every failed test.

Role	Owns	Should not routinely do
I3 · Execution Controller / Campaign Lead	Implementation management, debugging, integration, review, retries, checkpoints, and recovery within an authorized campaign.	Invent new product requirements or approve its own work as the final authority.
I4 · Execution specialists	Bounded code, research, tests, technical review, and source-linked evidence.	Override product decisions or claim strategic acceptance.

The product owner can reject an interface that feels confusing even when automated tests pass. A technical reviewer can reject incorrect implementation evidence. The Planner determines whether the authorized capability satisfies its strategic conditions. Those decisions must not silently replace one another.

In one common arrangement, ChatGPT helps the user reason about planning, while Codex edits and tests a repository. That is an example, not a product requirement or a mandatory subscription. A commercial service does not automatically become I2 or I3; those labels describe authority, obligations, and evidence requirements. One person or agent may perform several bounded roles, provided conflicts of interest and independent-review needs are respected.

3. What must remain accessible

When learning the real technical state, start with the source and environment that physically exist, then current canonical Product State, confirmed requirements, current campaign authority, DB and artifacts where relevant. A previous chat response—even one stating PASS—is weaker than current physical evidence. On the other hand, a new explicit decision from the CLIENT can change product intent; the system must then record that supersession rather than pretending an older interpretation remains authoritative.

Do not solve contradictory records by choosing whichever filename looks newer. An absent fact is UNKNOWN. A material contradiction between active sources of authority means HOLD for actions that depend on the disputed fact, followed by reconciliation. Read-only investigation can continue while material changes are paused.

The framework distinguishes three broad layers:

1. L0: evidence of what the CLIENT actually requested, clarified, or approved, including later changes.
2. L1: interpreted current requirements, product theory, and canonical Product State.
3. L2: implementation, code, databases, campaigns, checkpoints, reviews, and runtime evidence.

These layers prevent a persuasive conversational summary from being confused with deployed software. They also prevent a document containing a future plan from overriding a later, physically verified implementation result.

At minimum, a fresh worker should reconstruct the product objective, current requirements, verifiable code reference, relevant environment and DB witnesses, active cam-

paign if one exists, open gate, strongest evidence, unresolved unknowns, and next permitted or prohibited action. If this cannot be reconstructed, continuity is not yet demonstrated.

Terms you will encounter throughout the collection:

- **Scaffold:** the generated `.agentic-sdlc/` control layer. Its initial `HOLD / UNTRUSTED_CONTEXT` status is deliberate.
- **Discovery:** a read-only effort to understand the existing product, source, environment, and needs.
- **Adoption:** the reviewed promotion of verified facts and decisions into coherent control state. It is not blind copying from examples.
- **Product State:** the canonical description of the product's currently recognized capabilities and status.
- **Charter:** an authorized strategic contract for an engineering capability, including boundaries and acceptance criteria.
- **Gate:** a defined check that must be satisfied before a particular kind of acceptance. Different risks require different gates.
- **Currentness:** evidence that code, state, authority, and artifacts still refer to the versions actually being used.

4. How to use this manual

Read Chapter 01 before a first installation, then Chapter 02 before starting the first material feature. The remaining chapters are references to consult as their operational questions arise. You do not need to memorize eight chapters before trying the SMALL profile in a disposable or ungoverned project.

Each chapter has a different observable outcome:

Chapter	When to read it	What you should be able to verify
00 · Before You Begin	Before installing	You understand the responsibilities, boundaries, and overall lifecycle.
01 · Installation	When preparing a project	An installed, validated control layer exists; no adoption or execution authority is fabricated.
02 · Planner and Adoption	Before the first feature	Requirements, baseline, ownership, and next action can be reconstructed without the previous chat.
03 · Daily Operation	During active engineering	Work is authorized, executed, tested, reviewed, and recorded in the same campaign.
04 · Requesting Changes	When changing product behavior	The request describes observable behavior, priorities, boundaries, and open decisions.

Chapter	When to read it	What you should be able to verify
05 · Errors and Recovery	During incidents and uncertain state	Evidence is preserved and further damage is avoided while recovery is verified.
06 · Closing Capabilities	When someone says “done”	The exact candidate and applicable acceptance gates have sufficient evidence.
07 · Worked Club Example	To practice the whole flow	You can follow one member-management capability from requirements to acceptance.

After Chapter 01, a newly installed control layer may still correctly be on HOLD. After Chapter 02, a genuinely fresh agent should reconstruct the project without relying on the original conversation. After Chapter 06, the accepted candidate should be identifiable, with separate evidence for each applicable gate. If one of these outcomes is missing, return to the step that produces it. The number of documents you read is not itself evidence of installation or progress.

5. The product decisions only you can make

Before installing anything, write a short project brief. Explain the real problem, intended user, primary journey, data to be handled, expected usage environment, target date, budget, desired interface quality, and what is explicitly out of scope. If you have not made a decision, label it an open question.

For example:

> I want a local application to register club members and their dues. One administrative worker will use it. The first version will not accept online payments. It must retain payment history. I want a simple readable interface. The budget for paid services is zero.

You do not need to decide whether the implementation should use SQLite or PostgreSQL, how many branches Git needs, which framework to use, or which retry algorithm to implement. Engineering owns those choices, while explaining the consequences of decisions that affect the product or your reserved authority. With personal data, payments, publishing, or external integrations, the team must identify the relevant risks and obtain the permissions that belong to you. Agent involvement does not make consent, lawful processing, or publication rights automatic.

6. Scope, limitations, and versions

ArquitecturaRepo publishes reusable mechanisms and synthetic examples. Its transfer and recovery tests do not demonstrate that every application can be safely installed, operated, or shipped. The optional local `campaignctl.py` maintains certain campaign states; by itself it does not launch arbitrary coding agents or certify native Codex Desktop behavior. The public v1.4.0 support matrix distinguishes proven

portable behavior from unsupported host-specific claims.

The application, operating system, API price, model name, and UI of a third-party service can change. The manual therefore avoids making them permanent parts of the method. Always compare version-sensitive instructions to the exact release and the current code you actually installed. The original repository code and documentation use the MIT license; third-party dependencies and assets have their own rights and terms.

Git tags are version references, not proof that a hosting service technically enforces immutability. Verify release artifacts against their published digests, and distinguish any newer `main` documentation from the tagged distribution. A verified release does not authorize a new campaign, billable provider call, or product publication.

7. One complete example, from idea to acceptance

Imagine an association that wants to register members. The CLIENT decides who may register records, what information is mandatory, and what "deactivate a member" means. The Planner turns those decisions into testable requirements. It must not assume that deactivation means permanent deletion. Before authorizing development, it inspects existing code and state, identifies dependencies, and asks only the product questions that materially change the intended result.

Once the scope is confirmed, the Planner records current requirements, Product State, technical baseline, and a dated delivery plan. It chooses a capability that can be delivered and evaluated end to end. Its charter specifies the outcome, allowed boundaries, dependencies, relevant gates, and conditions that would invalidate the agreement.

The Execution Controller then organizes implementation. It may assign code, tests, review, and integration to specialists, or carry them out directly in a small project. It fixes ordinary defects and reruns relevant tests inside the same authorized campaign. It does not return every bug to the Planner as a new strategic decision. When a meaningful candidate exists, the Planner reviews the independent evidence and the CLIENT assesses reserved product and perceptual matters.

The cycle is not over because a convincing report was written or a large number of commits appeared. It is over only when the requested capability works to the accepted standard and the repository preserves enough current evidence for the next worker to continue. Chapter 07 follows this example in detail with fictional club data.

8. Common decisions and their owners

Situation	Decision and expected evidence
"Members in arrears cannot enroll in activities."	CLIENT defines the rule; Planner records the confirmed requirement and its tests.
"Should we use SQLite or PostgreSQL?"	Engineering chooses according to the verified usage, concurrency, backups, deployment, and risk.
"Can we publish our real member data in a demo?"	CLIENT must authorize any lawful, specific use; the team protects data by default.
"A test failed after a correction."	I3 diagnoses and repairs inside the same charter; evidence is rerun.

Situation	Decision and expected evidence
“I now want the local app to become a public web service.”	CLIENT confirms the product change; Planner reevaluates hosting, access, security, and scope.
“The coding agent said everything is ready.”	That statement is not acceptance. Require a source-bound candidate and the relevant technical and human checks.

9. A one-page project starter brief

Keep your first brief compact. A useful version includes:

- **Purpose:** what the first version solves and who will use it.
- **Main journey:** what the user tries to accomplish and how success is visible.
- **Data:** what is stored, whether it is personal, and required retention.
- **Environment:** device, operating conditions, connectivity, and offline needs.
- **Boundaries:** target date, spending limit, approved integrations, and exclusions.
- **Visible quality:** readability, accessibility, error messages, speed, and the experience you care about.

You do not need to prepare a full technical specification. The engineering process should discover the relevant details without inventing your unresolved product decisions. Keep the brief as the input to the conversation and formal adoption in Chapter 02.

10. How to know whether it is helping

After two or three delivery cycles, ask for a short demonstration: which new capability can you use, where is the evidence, what changed since the previous cycle, which material risk remains, and what is the next authorized action? If the answer is only “we made many commits,” “we ran more agents,” or “there is more documentation,” the effort has not yet been connected to a product outcome.

An architecture like this can reduce reliance on individual chat histories, but it does not eliminate judgment, security review, or real-world testing. Measure its value through continuity, clear ownership, usable capabilities, credible evidence, and reliable recovery—not the number of protocols or sessions.

Next: Chapter 01, Installation.

Sources and applicability

- Published v1.4.0 release
- Actual public support and first-user installation
- Release and upgrade boundaries
- Architecture overview
- Authority model
- MIT license

This is user guidance about the starter. No example or framework test is a substitute for verifying the adopter’s own product.

01 — Installation and Configuration

ArquitecturaRepo · Complete User Manual · v1.4 edition · English

> This chapter covers first-time installation and its boundary with project adoption. Commands follow the published v1.4.0 instructions. A successful scaffold is **not** evidence that the application has been understood, adopted, implemented, or accepted.

In this chapter

1. Environment and accounts
2. The starter, the fork, and the application
3. Verify the location before changing anything
4. Select a proportionate profile
5. Generate the control layer
6. Validate without manufacturing acceptance
7. The next action is discovery, not coding
8. Before attempting installation again
9. From scaffold to genuine adoption
10. Connections, access, and privacy
11. Safe troubleshooting

1. Environment and accounts

This walkthrough uses Windows, PowerShell, Python, and a local checkout of ArquitecturaRepo. GitHub can host your source; a conversational assistant can help with planning; a coding agent can work inside an authorized checkout. None of these particular commercial services is a universal prerequisite for the architecture. A developer comfortable with Git does not need GitHub Desktop, and a remote coding agent does not require a dedicated GPU in your own computer. You do need enough storage, a reliable way to access source code, and the permissions required by the tools you choose.

First identify the repository that contains the application you intend to develop. A frequent and damaging mistake is to open the framework's own folder in the coding tool and ask it to implement your application's feature there. The framework and the application have separate purposes and should normally have separate repositories. Decide which directory the tool will read and which directory it can modify before authorizing any write.

Check Python from PowerShell:

```
& "python" --version
& "py" --version
```

Python 3.12 is the version tested in the v1.4 portable continuous-integration runs. If neither command works, install an appropriate Python build from its official distributor, restart your terminal, and repeat the version check. Do not run an untrusted installer or assume a command worked when it returned a nonzero exit status. The

UI location of GitHub, ChatGPT, or a coding tool may change; identify the operation you need rather than relying on the color or position of a particular button.

Protect your accounts. Enable suitable account authentication, work with the least necessary access, and keep tokens, private keys, and connection strings out of prompts and the repository. Connecting a provider to a chat does not prove the assistant has read the correct repository or has permission to operate on the actual application. The presence of a tool is different from authorization to use it.

2. The starter, the fork, and the application

Three things are easily confused:

Location	Purpose	What it must not be mistaken for
`ArquitecturaRepo/`	The public source, protocols, templates, tools, tests, and documentation used to create a control layer.	Your application's source or Product State.
Your optional fork	Your own version of the public starter if you need to maintain custom changes.	A newly created application.
`club-app/`, or your real application repository	The actual product, its code and data, and its installed `.agentic-sdlc/` control layer.	A disposable copy of synthetic examples.

The starter's examples deliberately contain illustrative facts. Copying one of their Product State files into a real application would silently assert made-up code references, requirements, owners, and decisions. Installation therefore creates a cautious initial state rather than declaring a valid adopted project.

A reasonable local directory arrangement is:

```
C:\Proyectos\
    ArquitecturaRepo\
    club-app\
```

From the source repository, read its README and MIT license. If you want to maintain your own starter variant, fork it; otherwise an ordinary checkout is enough. Create or select the actual application separately. A real existing application does not need to be erased and recreated merely to follow this guide. Record the exact version or tag of the starter used for installation, because `main` may subsequently move.

For a source archive, prefer the v1.4.0 release, which provides the versioned archive and its SHA-256 checksum and manifest. Verify the artifact before using it. A Git tag and a published ZIP are distribution identifiers; they are not permission to run code in a different repository. Be aware that any release notes published after the ZIP was built do not retroactively change the ZIP's contents.

3. Verify the location before changing anything

Open a terminal in the starter checkout. These PowerShell commands only inspect your current working location, the available interpreter, and Git state:

```
Get-Location
& "python" --version
& "git" status --short
```

If the Git command reports that no repository exists, investigate the path before proceeding. If it lists local changes, do not discard them simply to imitate a tutorial. Identify their owner and role, and preserve them when relevant. Confirm the target path separately; the scaffold command operates on the path you pass it, not on the application you merely intended.

If using GitHub Desktop, ensure the selected repository is the application when you intend to work on its implementation. If using a coding agent, open the application's exact working directory and give it the starter location as a read-only reference until the installation boundary is clear. The agent may perform installation if authorized, but should first inspect the starter, the destination, and any existing governance in read-only mode.

If the application has history, migrations, live data, existing governance, or an `.agentic-sdlc/` directory, stop and examine its current state. Removing an old checkout or overwriting its control layer can erase the evidence needed for safe recovery. **For an already-governed application, use the existing-governance attachment preflight, not the generic scaffold.** That route is read-only and does not promise an automatic migration from v1.2 or v1.3.

4. Select a proportionate profile

Choose the profile by product risk and operating characteristics, not by how many AI agents you hope to use. The framework defines six public profile families in profiles/profiles.json:

Profile	Choose it when the product has…
`SMALL`	A simple product boundary without material state or operational risk that requires heavier controls.
`STATEFUL`	Persistent data, users, backups, schema changes, recovery, or a runtime whose state matters.
`ARTIFACT_HEAVY`	Numerous or costly outputs, lifecycle and lineage requirements, storage pressure, or rights-sensitive artifacts.
`MULTI_WORKSTREAM`	Separately planned workstreams contributing to one final product, with integration and interference to coordinate.
`MULTI_PRODUCT`	Distinct products sharing resources, policies, or infrastructure; may justify an I1 Master Owner.

Profile	Choose it when the product has…
`HIGH_CONSEQUENCE`	Material consequences for privacy, security, finances, health, integrity, or irreversible external operations.

`HIGH_CONSEQUENCE` is not a prestige label, and `MULTI_PRODUCT` does not automatically imply you must create many separate agents. The profile selects reusable control families; it does not prove your product's actual risk, create technical evidence, or grant authority to publish, spend, or process personal data. Document why you selected a profile and revise it when verified circumstances change.

The fictional club application used in Chapter 07 is `STATEFUL`: it stores member records, payments, and activity enrollments. It does not thereby need an elaborate Master Owner hierarchy or a large agent fleet. A small version may still have a single human owner and a small engineering arrangement while implementing the state and recovery controls appropriate to its data.

For first exploration in a truly new or ungoverned project, `SMALL` is the recommended baseline. Start with the least complex arrangement that can safely represent the facts and risks. Escalating later should be a reasoned adjustment, not a reaction to the fact that a protocol mentions a more elaborate role.

5. Generate the control layer

From the checked-out v1.4 starter, a new project can be scaffolded with the following PowerShell command. Use a destination path you have already verified:

```
& "python" "tools\scaffold_project.py" --target "..\mi-proyecto" --profile "SMALL" --p
```

For the fictional club application, a justified stateful choice would look like this:

```
& "python" "tools\scaffold_project.py" --target "..\club-app" --profile "STATEFUL" --p
```

Do not combine command lines into one and do not paste the line breaks of printed sample layouts into a terminal. The path, objective, product ID, and language are examples to replace when you run an installation, not assertions about an actual project.

The installer creates a new repository-local `.agentic-sdlc/` layer in the destination. It does not rewrite existing application source or workflows. It must refuse to overwrite an existing canonical control layer; do not use a force option as an ordinary recovery technique. A second run is not a substitute for understanding a failed first run. If the installer prints the next read-only commands, prefer those commands for the exact target and version you actually used.

The initial status intentionally is `HOLD / UNTRUSTED_CONTEXT`. This means the template does not yet know your code baseline, current Product State, requirements, owners, or active campaign. Editing a status field to `READY` would falsify continuity. There should be no invented project acceptance or authorized coding campaign at this stage.

The optional `--with-runtime` flag adds reference campaign machinery; it does not install an external model, authenticate a provider, start native Codex Desktop jobs, or authorize any work. The optional GitHub Actions feedback scheduler is disabled by `--no-feedback-workflow`; this does **not** disable the local metadata report generator. The report configuration is a separate choice discussed in Section 10.

6. Validate without manufacturing acceptance

Move into the target application's directory after a successful scaffold. Install the declared local validation dependencies and run the three v1.4 commands:

```
& "python" -m pip install -r ".agentic-sdlc\requirements-validation.txt"
& "python" ".agentic-sdlc\tools\project_status.py" --control-root ".agentic-sdlc"
& "python" ".agentic-sdlc\tools\discover_project.py" --project-root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py" --root ".agentic-sdlc"
```

The status command reports recognized governance and what remains unresolved. Discovery inventories file metadata and hashes without copying source contents into a remote service; it refuses links outside the intended project and linked destinations used for output. Reconstruction checks whether a fresh worker could recover meaningful, source-bound current state. A failure or HOLD in the last command can be the **correct** result before adoption. It should identify facts that still need confirmation instead of hiding them.

The installed control layer also includes structural validation tools. The portable release guide's three commands are the appropriate starting sequence because they expose currentness and adoption needs, not just well-formed JSON. Tests of control contracts do not demonstrate that the club can register a member, that its database is safe, or that a user will understand its interface.

Confirm three observable facts: `.agentic-sdlc/` exists at the intended target; the application's existing files and workflows remain intact; and the initial status and discovery findings can be read without relying on a chat. Save any error output for diagnosis. A missing dependency such as `jsonschema` is a local environment problem; install the declared requirements rather than altering project records or creating another checkout.

Completion of this step means "scaffold created and inspected," not "application adopted." The distinction is central to the framework: installation checks the mechanism, while adoption connects that mechanism to the real product.

7. The next action is discovery, not coding

After installation, follow the installation/adaptation prompt, product discovery prompt, and the transfer and adoption contract. If the application already existed, the reanchoring prompt is relevant. These are source-backed instructions for inspecting the actual project; they are not permission to let an agent make unresolved product decisions.

The Planner should determine what the product currently does, what the CLIENT wants changed, what code and data are current, what environment matters, and which facts require confirmation. A material implementation campaign should wait for an adopted baseline and an explicit charter. Coding immediately after creating the directory skips the step that makes the governance truthful.

8. Before attempting installation again

If `.agentic-sdlc/` already exists, inspect it before rerunning anything. It may contain canonical Product State, an active campaign, a lease, current ownership, DB references, or evidence that must survive. A successful-looking second scaffold can still

cause serious continuity damage if it replaces such material.

Do not use deletion, force-overwrite, or a fresh repository as a first response to a validation failure. Classify the failure: environment or dependency, path, schema, unsupported compatibility, reference/currentness, or an actual conflicting control layer. Keep the complete error report and ask the responsible engineering owner to correct the cause in place when possible. For a genuinely governed project, the read-only preflight is the default safe path.

9. From scaffold to genuine adoption

The transition to an operational control layer has several distinct parts:

1. Inspect the real application and its environment, initially without writing.
2. Ask the CLIENT to confirm or correct the intended user behavior and scope.
3. Record traceable, current requirements and the technical/source baseline.
4. Construct the delivery plan and a concrete adoption specification.
5. Check ownership, files, links, DB state and dependencies in preflight.
6. Promote internally consistent control state using the supported adoption process.
7. Validate the resulting records and reconstruct them in a fresh context.
8. Only then route a materially authorized capability to its engineering owner.

The adoption guide explains this boundary. The assistant must not fabricate a JSON baseline, current SHA, active Goal, or approval to make a validator go green. If the necessary fact is a product decision, it belongs to the CLIENT. If it is technical, the appropriate engineering owner must investigate and demonstrate the result. “We are ready” should mean the evidence and authority are reconstructible, not merely that an installation script exited successfully.

10. Connections, access, and privacy

You may connect GitHub or another provider to assist read-only exploration. Begin with only the repositories and rights needed. A signed-in service is not proof that the correct repository, branch, application, or runtime is active. Before allowing a coding tool to write, check its current directory, configured permissions, and relevant branch or worktree. Do not grant global access “just in case.”

Local reversible checks differ sharply from publication, destructive filesystem operations, API purchases, sending project content to third parties, or modifying production services. The latter need explicit authority with a clear target and limit. Do not copy secrets into agent prompts, public issues, Git history, workflow logs, or the public starter. The application can remain private while the governance starter is public.

The framework can generate privacy-preserving metadata feedback locally. In v1.4 the local strict-metadata report configuration is enabled by default, while external collection is not bundled. `--no-feedback-workflow` removes the periodic workflow scheduler; it does **not** disable the local generator. To disable reporting entirely, set `enabled=false` in the installed feedback configuration. Review the feedback privacy policy and any executable workflow changes before making an affirmative choice.

11. Safe troubleshooting

Observation	Safe interpretation	Next action
Python is not recognized.	The terminal cannot locate an interpreter.	Verify your installation and path; leave the application unchanged.
`.agentic-sdlc/` is absent.	The scaffold may have targeted another folder or failed.	Compare the exact target path and command output.
`HOLD / UNTRUSTED_CONTEXT`.	First installation succeeded without inventing project facts.	Continue discovery and adoption; do not suppress HOLD.
`jsonschema` is missing.	Validation dependencies were not installed in the active interpreter.	Install the declared requirements and rerun the read-only checks.
An existing control layer is preserved.	The installer protected canonical governance.	Use the read-only governed-project attachment procedure.
Validation or reconstruction reports a conflict.	Structure, authority, source, or DB witnesses may disagree.	Preserve the report; reconcile the exact cause before material writes.
The coding agent sees another repository.	Its operational scope points to the wrong directory.	Stop writes and verify target, permissions, and working tree.
Tests pass but the application does not open.	Control tests did not cover the application's physical behavior.	Run the actual product and check its real user journey.

The correct result of the installation chapter is an initial control layer whose structure and limits can be inspected. Its adaptation to real code, requirements, data, and current authority is the subject of Chapter 02. Do not treat those two milestones as interchangeable.

Next: Chapter 02, Planner and Adoption.

Sources and applicability

- v1.4.0 GitHub Release
- Public start and supported boundaries
- v1.4.0 release and upgrade notes
- Fork and bootstrap
- Transfer and adoption
- Existing governed-project attachment
- Executable validation
- Feedback and privacy
- Scaffold source

All instructions concern the starter or a fictional application. They do not prove installation, acceptance, or runtime compatibility for a specific adopter's project.

02 — The Planner and Project Adoption

ArquitecturaRepo · Complete User Manual · v1.4 edition · English

> A scaffold is a starting control layer, not a project baseline. This chapter explains how the Planner reconstructs an application, obtains the CLIENT's product decisions, adopts verified state, and authorizes the first bounded campaign. For concrete commands use the v1.4 public start and adoption contract.

In this chapter

1. A conversation is not the project
2. Prepare the working space
3. Discovery: ask the right questions
4. Technical baseline and delivery plan
5. Adoption: connect the starter to reality
6. The real examination: a fresh context
7. The first authorized engineering campaign
8. An example first Planner session
9. A baseline the CLIENT can actually confirm
10. An example of a first campaign

1. A conversation is not the project

The I2 Planner owns strategic decisions for a product or bounded workstream. A chat application can provide a convenient interface, but its conversation history is neither a canonical database nor evidence that the latest implementation still matches an earlier statement. Durable authority comes from the CLIENT's current requirements, applicable protocols, verified repository and DB state, active ownership, and valid campaign records. If the last executor said "done," the Planner still needs evidence of the exact candidate and the relevant acceptance gates.

In a small project, one Planner and one execution owner may be sufficient. An I1 Master Owner is useful when several products, separately planned workstreams, shared budgets, or cross-cutting operational controls demand an additional systemic owner. It is not a higher-ranking replacement for the product Planner, and simply naming another chat "Master Owner" does not improve delivery.

The Planner thinks from the intended product outcome backward: who will use the capability, which current interfaces and data it affects, how the result is observed, which dependencies are real, and what would falsify acceptance. It then prepares a capability-level charter. Within that authorized scope, the I3 Execution Controller handles debugging, retries, code organization, integration, review and recovery. An ordinary bug should not create a new charter or a new strategic planning cycle.

2. Prepare the working space

Start with the actual application repository, not a remembered description. Read the project's `AGENTS.md`, applicable governance protocols, current requirements, canonical Product State, relevant source references, and existing campaign or runtime records. If the Planner cannot see a necessary repository or host surface, it must identify that specific gap. A plausible memory of a previous conversation is not an acceptable substitute.

You may organize conversations in a ChatGPT project or another collaboration space and attach the manual for convenience. If using a GitHub connector, restrict it to the repository and permissions required. Verify actual access with a read-only request for a known current file. A connected account does not guarantee access to every branch, DB, issue, or runtime. The architecture does not mandate any model, reasoning setting, subscription, or particular vendor.

A useful opening role instruction is:

> Act as this project's I2 Planner. I am the CLIENT: I decide product requirements, priorities, and reserved external permissions. First reconstruct the actual repository and applicable protocols; do not invent state. Define capability boundaries and charters. Assign tactical execution, debugging, and recovery to I3, within authorization. Do not transfer routine Git, testing, source-currentness, or engineering-recovery decisions to me. Clearly distinguish what is verified, what remains unknown, and what product choice requires my answer.

This is a role entry point, not an override for `AGENTS.md`, the current requirements, or an already authorized campaign. A fresh session still needs to read the canonical surfaces. A role title grants no additional filesystem or provider permission by itself.

3. Discovery: ask the right questions

Begin by identifying the user and the problem, not by proposing a technology stack. Ask for complete user journeys, the information that must persist, important error cases, connectivity, exposure to the public Internet, personal-data sensitivity, cost constraints, dates, accessibility expectations, and explicit exclusions. An effective Planner can group three to seven related product questions at a time, then return a coherent summary for correction.

Questions should affect meaningful product behavior:

Useful question for the CLIENT	Technical decision engineering should own
Who uses the system, on what devices and from where?	Ports, server topology, UI framework, and deployment mechanics.
What happens to payment history when a club member is deactivated?	Data schema, foreign keys, migrations, and transactional operations.
Are online payments part of the first version?	Payment-provider design, subject to separate authorization for spending and handling real transactions.
What usable result matters before the target date?	Internal task order, test implementation, checkpoints, and code organization.

Useful question for the CLIENT	Technical decision engineering should own
Should more than one staff member edit a record simultaneously?	Concurrency strategy and conflict handling consistent with the requirement.

At the end of discovery, present one product baseline that the CLIENT can verify: functions, user flow, data rules, nonfunctional quality, security and network exposure, interface behavior, backups and recovery needs, costs, dates, exclusions, and open questions. A CLIENT confirmation is not a legal signature or a guarantee that requirements will never change. It establishes a traceable current decision that later revisions must explicitly supersede.

Avoid pushing purely technical uncertainty onto the user. “Which database should I choose?” is usually not a product question. “Can staff lose the last hour of payments after a power cut?” absolutely is, because the answer constrains technical recovery design. The Planner should translate technical tradeoffs into their impact on the user before requesting a reserved decision.

4. Technical baseline and delivery plan

After confirming product intent, the Planner chooses or confirms a technical strategy against the actual code and constraints. It covers language and framework, persistence, schema changes, deployment, secrets, access control, testing, UI integration, backups, and recovery. In an installed control layer, current technical choices belong in `state/TECHNICAL_BASELINE_CURRENT.json`, not solely in a conversational recommendation.

The delivery plan records milestones with observable completion conditions, target dates, dependencies, confidence, risks, and next review dates. A revised date must be traceable; silently overwriting an old promised date erases information necessary to reason about delivery. A checklist of programming tasks without dates or user-visible outcomes is not an adequate delivery plan.

The starter contains `tools/baseline_guard.py`, which checks whether the material prerequisites for a campaign are coherent. It must not be bypassed by writing invented requirements or status fields. A campaign should remain blocked if the CLIENT has not confirmed its product baseline, if a material security or network decision is unresolved, or if source/authority state is contradictory.

The Planner should also examine how the capability will actually be executed. A charter explaining only “implement member management” may be strategically meaningful but too vague for a long-running, costly controller. The outcome-first execution contract introduces a bounded work order for multi-operation, paid, batch, or substantial-runtime campaigns. The charter establishes what and why; the work order ties the known inputs to an executable method, a budget, concrete output, stop conditions, and physical evidence. It is **not** a second campaign.

The portable `tools/work_order_guard.py` can `validate` a work-order contract and `assess` progress receipts. Its assessment is a decision aid, not a physical or perceptual acceptance gate and not authorization for a vendor API. A report saying “review-ready” does not establish that the product is usable. If a deterministic file operation can be done with filesystem tools, there is no reason to require paid model evaluation for every file.

5. Adoption: connect the starter to reality

The adopted state must describe the actual application. The reproducible workflow begins with the scaffold, a source inventory, a reviewed adoption specification, preflight on a temporary control copy, and a journaled promotion of coherent records. `discover_project.py` collects file metadata and hashes; it does not decide the product's purpose. `apply_adoption.py` materializes decisions and verified facts described by the adoption specification; it cannot discover missing user intent by itself.

The adopted result should connect current requirements, source references, technical baseline, Product State, bootstrap context, owner registry, and any project overlay. Where a DB or SQLite WAL witness matters, its state and reference must be consistent. An existing campaign must not be reset or silently replaced to simplify adoption. In a project with an established canonical control layer, use the read-only governed-project preflight. Generic reinstallation is not a v1.2/v1.3 upgrade mechanism.

If the Planner has read-only remote visibility while the I3 controller has an authorized local checkout, the Planner can prepare one complete execution dispatch. The CLIENT should not have to hand-edit ownership registers, reconcile current SHA values, or assemble fragments of JSON. The executor works from the exact source and reports physical results. The original application's source, history, data, and workflows must be preserved unless an independently justified and authorized change is within scope.

The adoption report should answer five concrete questions:

1. Which repository and source version were actually inspected?
2. Which requirements and decisions did the CLIENT confirm?
3. Which current technical state, owners, and control records were materialized?
4. Which validations were physically executed, with what result?
5. Which unknown, HOLD, or first permitted work boundary remains?

If the answer is only "files installed" or "all green," the transition is not yet demonstrated. The tools are scaffolding for truthful control state, not a way to impersonate genuine product evidence.

6. The real examination: a fresh context

Open a genuinely new working context, with no hidden reliance on the previous conversation's narrative. Give it the repository identity, authorized read access, and the normal entry instructions. Ask it to reconstruct its role and limits, current requirements, source ref, Product State, active campaign and checkpoint if any, open acceptance gate, best evidence, unresolved unknowns, next allowed action, and prohibited actions.

A successful answer must cite current durable sources. Merely repeating the conceptual architecture from memory fails the point of the test. It also fails if the new worker treats a superseded document as current, invents DB state, confuses an executor PASS with final acceptance, or asks the CLIENT to fix a technical currentness conflict the system's owner should resolve.

If access to the repo was missing, classify it as an access problem and repair it within authorization. If the repository's own records contradict one another, identify the canonical owner and repair the durable source or currentness mechanism. Do not "pass" the test by handing the new chat a longer summary of the old conversation. The independent reconstruction method is explained in the fresh-context test and fresh-

context probe prompt.

7. The first authorized engineering campaign

Choose a capability that can be implemented and evaluated end to end. Its charter should state the product result, dependencies, ownership, affected interfaces, interfering resources, relevant gate types, evidence expectations, and conditions that would falsify acceptance or invalidate the charter. It should not prescribe twenty tiny commits or how many retries I3 may need for ordinary defects.

Do not equate "the control layer validates" with "the member can register and recover a payment." The first is a framework-contract check; the second needs application integration, persistence, behavior, possibly security and usability evidence. A sensible charter distinguishes unit, integration, DB/currentness, semantic/generalization, physical, perceptual/human, and release gates according to the real risks. Passing one does not silently cover the others.

Once authorized, I3 owns implementation and in-scope rework continuously. Code defects, failed tests, reviewer findings, retries, internal refactors, migrations within the envelope, and recovery are not reasons to send every iteration back to the Planner. A material change in product intent, authority, critical interface, or strategic scope may invalidate the charter; that is a different event from a form validation bug.

8. An example first Planner session

Use a short, outcome-focused opening request, without asserting that adoption is complete:

> I want an application to register club members and dues. Before planning implementation, inspect my actual application repository and the ArquitecturaRepo starter in read-only mode. Guide me through product decisions that remain open. Separate confirmed choices from assumptions; do not fabricate application or infrastructure facts. Prepare one product baseline for my review. Once I confirm it, select the justified profile and technical strategy, produce a dated delivery plan, and arrange adoption. Do not launch material implementation while baseline and continuity gates remain unsatisfied.

The Planner should inspect existing files, branches, and state before asking questions it can answer by reading. It may ask how many people will use the system, where they work, what historical payments must be retained, or how a late payment differs from a cancelled one. Each clarification should change the product contract or a significant constraint, not merely extend the conversation.

9. A baseline the CLIENT can actually confirm

"Build a professional CRUD" is not a reviewable product baseline. A compact example can be much more precise:

Dimension	Proposed first-version requirement	Decision status
Primary user	One administrative worker on a club computer.	Awaiting CLIENT confirmation.
Data	Members, dues status, payment history, activity enrollments.	Awaiting CLIENT confirmation.
Retention rule	Deactivating a member must not delete earlier payments.	Awaiting CLIENT confirmation.
Member operations	Create, find, edit, deactivate and inspect history.	Awaiting CLIENT confirmation.
Security and network	Initially local; no public Internet deployment.	Awaiting CLIENT confirmation.
Interface	Readable forms, useful search and understandable errors.	Awaiting CLIENT confirmation.
Recovery	Define backup interval and acceptable loss of recent records.	Open product decision.
Exclusions	No online payments or mobile application in version one.	Awaiting CLIENT confirmation.

Only mark a line confirmed when the CLIENT has actually agreed. When the baseline matches their decisions, record the confirmation and its version in the appropriate canonical project state. If the user later changes a requirement, record the supersession and its impact; do not edit away the earlier decision as though it never existed.

The Planner then records technical choices and delivery milestones, including dates, dependencies, acceptance evidence, confidence, and risk. Those technical decisions should be justified by this product, not by fashionable stacks or whichever language an assistant happens to prefer.

10. An example of a first campaign

The first club campaign could be: Create, find, edit, and deactivate members while preserving historical payments and showing clear errors for invalid input. The result can be demonstrated in a running application and verified against persistent state. The charter identifies its product boundary, data rules, dependencies, and relevant gates; it need not dictate every file or every terminal command to the executor.

I3 receives that authorized result and current source. It implements, tests, reviews, integrates, and repairs defects inside the campaign. If the form rejects a valid phone number, I3 fixes and retests it. If the CLIENT instead changes the product to serve hundreds of clubs over the public Internet, the product requirement and architectural consequences belong back with the CLIENT and Planner before strategic rerouting.

At the end, the candidate is identified by source and evidence. The Planner can assess technical and strategic gates, and the CLIENT can exercise the actual member journey and accept or reject the reserved perceptual/product aspects. A successful test fixture does not replace either real runtime evidence or the CLIENT’s experience.

Next: Chapter 03, Daily Operation.

Sources and applicability

- Product discovery, requirements and delivery
- Installation and adaptation prompt
- Product discovery session prompt
- Fresh-context probe
- Fresh-context test
- Transfer/adoption contract
- Existing-governance attachment
- Outcome-first execution work orders
- Planner operating protocol
- Baseline guard source
- Work-order guard source

The examples describe synthetic club records. None is evidence that a particular target repository has been inspected, adopted, or accepted.

03 — Daily Operation

ArquitecturaRepo · Complete User Manual · v1.4 edition · English

> This chapter follows an already adopted project with a valid current charter. The framework's optional controller provides portable reference mechanisms; it is **not** a promise that Codex Desktop, a cloud agent, or another proprietary host will execute commands autonomously. Runtime claims must be verified on the actual host.

In this chapter

1. Start a session without starting over
2. Turn a wish into a verifiable capability
3. What I3 does when work becomes difficult
4. Reserved authorizations and external actions
5. Read an executor report critically
6. Git and continuity
7. The five-question daily closure
8. A working day without depending on chat memory
9. An example of a bounded dispatch
10. Interpret controller states
11. Process the final report
12. Leave a recoverable session

1. Start a session without starting over

Before sending a new programming request, identify the actual project and worktree, current branch and source reference, unresolved local changes, canonical Product State, the active charter if one exists, and the last verified candidate. A worker returning after a host session ends should read the durable checkpoint. Restarting a similar-looking project from an old prompt is not continuity.

When the checkout is trustworthy and the baseline is known, look at the exact relevant Git delta and its dependencies. Do not automatically reclone or blindly `git pull` every time an agent resumes; those actions can disrupt unintegrated user work. Conversely, do not trust incremental sync when the source diverged or current authority is unresolved. Read-only reconciliation comes first in that case.

The CLIENT should receive a compact current-state summary without having to choose which SHA or DB snapshot is authoritative. It is an engineering responsibility to inspect the actual repository, the project instructions, dirty working tree, and relevant leases or current-state records. The Planner should recover the product objective and the authorized campaign state before routing a new feature.

A split-screen arrangement with one planning chat and one coding environment may be convenient, but it is not required. A reliable handoff consists of source-bound instructions, canonical state and evidence, not the position of application windows. If the required evidence cannot be read, mark the specific missing fact UNKNOWN and stop only the actions that depend on it.

2. Turn a wish into a verifiable capability

“Register dues payments, retain history, and show whether a member is in arrears” describes observable behavior. “Write the payment function” describes only a possible implementation step. The Planner should start from the former, identify existing interfaces, data rules, dependencies and risk, then authorize a bounded capability. I3 decides the internal sequence of code edits and tests within that charter.

For example, the CLIENT can say:

> I need staff to register dues payments without losing historical records and to see who has paid up. Do not add online payments yet, and keep the screen simple. I want to test the complete workflow before accepting it.

This is enough to establish intent, but not necessarily enough to begin a campaign. The Planner still needs to verify current requirements, data retention, deployment boundaries, source and ownership. The resulting dispatch names the expected capability, source inputs, limitations, affected interfaces and owners, execution gates, and a precise review-ready output. It does not ask the CLIENT to coordinate specialists or select technical retry strategies.

For long-running or batch-heavy work, the outcome-first work order makes execution operational: source-pinned inventory, ordered available-tool route, budgets, meaningful stop conditions, concrete output and physical readback. A charter and a work order are complementary: the first defines strategic authority; the second helps I3 use that authority efficiently. Neither can authorize an external bill or private-data disclosure by implication.

3. What I3 does when work becomes difficult

Suppose the implementation exists but the integration test fails after a payment update. I3 preserves the observed failure and environment, traces the affected component, fixes the cause, runs the relevant tests again, integrates the candidate and obtains an appropriate independent review. The problem remains in the original campaign unless it reveals material new scope or authority.

In-scope debugging, refactoring, retries, migrations within the charter, evaluator repair, integration and crash recovery belong to I3 continuously. Repeated failure of the same class is a reason to inspect the subsystem and its test design as a whole, not to stack another local patch or return to the Planner for a new microcampaign. The Planner should examine a consolidated terminal candidate, not every tactical iteration.

An unresolvable source/currentness conflict, an absent permission for an external action, or an actual change in intended product behavior requires a different response. I3 must describe the precise blocker and causal owner instead of disguising a new permission as a retry. Read-only diagnosis can often proceed while writes remain on HOLD.

Independent review must cover the exact candidate offered for strategic audit. If the code, schema, or relevant artifact changes materially after a favorable review, that earlier result is not blanket approval of the new candidate. Record the new evidence and rerun checks affected by the change; do not invalidate unrelated verified work without a reason.

The optional `tools/campaignctl.py` supports reference campaign control in the portable distribution. It is not a universal autonomous coding runner. Similarly,

Goal and tactical-plan helpers can manage records without proving that a particular host has activated native `/plan` tools or write-safe dispatch. The reference runtime describes its supported boundaries; claims about a real agent host need separate verification.

4. Reserved authorizations and external actions

Different operations carry different permission and risk profiles:

Action	Appropriate treatment
Inspect a repository or run local tests within granted scope.	Ordinary authorized technical investigation.
Edit and integrate source within an approved charter.	I3 engineering responsibility, subject to the exact worktree and ownership.
Install a new dependency or open a new tool.	Check provenance, necessity, permissions, and the effect on the project.
Deploy, publish, send messages, spend provider credit, or delete material data.	Obtain the specific reserved authorization and a proportional preflight.
Paste a secret, password, or private customer export into a prompt.	Do not do it; use the project's approved secret/data mechanism.
Encounter contradictory canonical DB, ownership, or source witnesses.	HOLD dependent writes; reconstruct and reconcile current state.

The CLIENT decides product behavior, acceptable risk, external publication and spending boundaries. The CLIENT should not have to resolve merge conflicts, pick database index names, repair migrations, or coordinate recovery. Some product choices cannot be delegated to engineering; many technical details can and should be.

5. Read an executor report critically

A useful report identifies the exact source candidate and environment, the intended behavior, what changed physically, which tests and demonstrations ran, what failed and how it was corrected, what independent review examined, and which gate remains open. The number of commits, tokens, agents, checkpoints or runtime hours is not evidence of a usable feature.

If a report says "PASS," ask which gate passed. A unit test may exercise a calculation with mocks; an integration test may connect components; a DB/currentness check may show that the persisted record survives an actual transaction; a physical check may open the real application; a perceptual check may confirm that a human can use it. Passing one does not silently cover the others, and a release gate has its own scope.

The report should also name the next action and its owner. A new business requirement belongs to the CLIENT and Planner; a known in-charter bug remains with I3; a complete source-bound candidate returns for consolidated strategic audit. This avoids turning a nontechnical product owner into the incident coordinator.

When accepting a capability, the Planner records which candidate was accepted, against which current charter and evidence, and what can be done next. When

rejecting in-scope defects, it returns one consolidated list of falsifiers to the same campaign. A genuinely changed product intent must be recorded as a traceable decision, not hidden under “bug fix.”

6. Git and continuity

Git stores code and history; it is not itself a complete Product State or authorization ledger. A commit identifies content and parents, not every human or agent responsible for a decision, not a real DB witness, and not final product acceptance. A branch advancing does not demonstrate that a particular worker had authority or that a capability is current.

Do not create a canonical commit for every worker message. Execution specialists should provide their bounded changes and evidence to the integration owner, who checks interactions and currentness. A worktree may contain valuable uncommitted changes; never discard them simply to make a status check show a clean tree. When several workers share files, DB state or runtime surfaces, resolve interference based on affected resources rather than project name alone.

An existing correct checkout is normally preferable to another clone. Reuse it when safe, and use temporary worktrees only when justified by genuine concurrency. Keep generated screenshots, exports, renders and caches in ephemeral or ignored locations unless they are required evidence. Protect disk space as a production resource.

After a session limit, the same charter can continue if the current source, checkpoint, owners and evidence are reconstructible. A vendor’s token limit is not a Charter Invalidation Event. When a host cannot resume a specific thread, use the canonical repository state rather than inventing continuity.

7. The five-question daily closure

At the end of a work period, obtain answers to five questions:

1. Which capability is active, and who currently owns execution?
2. Which code reference, Product State and relevant DB/currentness witness have actually been checked?
3. Which was the last real gate passed, and what evidence belongs to it?
4. What defect, risk, permission or product decision remains open?
5. What is the next legal action tomorrow, with a named owner, without requiring the CLIENT to arbitrate technical state?

Material decisions and next routes should be persisted in the repository’s canonical locations. A chat may be useful discussion context, but it must not be the only surviving statement of current truth. The next worker should be able to answer the same questions by reading the project.

8. A working day without depending on chat memory

At the beginning of a day, recover the existing goal before proposing another feature. The Planner checks whether a campaign is still active, its integrated source reference, missing evidence and reserved decisions. The possible outcomes are straightforward: continue I3 execution, audit a consolidated terminal, request a product decision, or authorize a genuinely new capability after its prerequisites are satisfied.

A session interrupted by usage limits or a crash should resume from an actual recorded checkpoint. If no checkpoint exists but durable state is sufficient, reconstruct it. If neither is sufficient, identify the precise lost evidence and HOLD dependent changes. A fresh chat repeating the words “we were halfway through” is not evidence of the halfway state.

9. An example of a bounded dispatch

The following message illustrates the content of an authorized work order. It is not a universal permission to run in any repository:

> **Goal:** Implement the current charter’s member-management capability.
>
> **Context and inputs:** Read the real `AGENTS.md`, confirmed requirements, canonical Product State, active charter and current source. Reconstruct applicable ownership and currentness before writing.
>
> **Constraints:** Retain payment history; preserve the confirmed local-only exposure; do not add online payments or unrelated features; do not publish or spend on external services.
>
> **Execution gates:** Physically exercise create, search, edit, deactivate and persisted history after restart; test invalid and duplicate inputs; independently review the integrated candidate.
>
> **Done when:** Provide the exact ref, source-bound evidence, review result, residual risks and an honest terminal for Planner audit. Resolve ordinary bugs, retries and internal refactors within the existing charter.

The actual dispatch should be adapted to the current approved charter and available tools. Do not invent `AGENTS.md` or Product State just because their names appear in this example. Locate the real surfaces and report any genuinely absent ones.

10. Interpret controller states

Status labels are operational clues, not substitute acceptance:

State	What it indicates	What it does not prove
`READY`	A runnable unit has satisfied prerequisite checks.	It is already executing or will succeed.
`ACTIVE`	An authorized unit is owned by an active execution context.	The resulting application behavior is correct.
`VERIFY`	Applicable checks are being assembled or executed.	Every product gate is satisfied.
`HOLD` / `CAPACITY`	A recorded dependency or resource condition blocks progress.	A new campaign must be opened.
Tactical `DONE`	A bounded execution unit met its local conditions.	Planner or CLIENT accepted the capability.

State	What it indicates	What it does not prove
Strategic terminal	I3 has returned a candidate or a precisely evidenced blocker.	Publication or a reserved external action is authorized.

The portable reference implementation does not create a complete distributed agent environment. If a project uses another controller, require equivalent verified guarantees of authority, currentness, concurrency, journal recovery and gate separation in the actual implementation.

11. Process the final report

The Planner reviews the exact candidate, proven environment, current requirements, actual tests, independent review and open risks. It should not redo I3's tactical implementation or rubber-stamp a self-reported PASS. Where evidence is incomplete, name the missing gate instead of treating all gates as failed or passed together.

For an in-scope rejection, the return message should give observed behavior, reproduction conditions, the exact evidence and the blocked gate in one consolidated batch. I3 owns correction and integration. If the same subsystem repeatedly fails for the same reason, require broader subsystem hardening and independent review within the campaign rather than serial one-line patches.

A truly new product rule, permission, or incompatible interface may require strategic reentry. That is not an excuse to reopen planning for a known regression. The distinction protects both product coherence and delivery speed.

12. Leave a recoverable session

Before stopping, persist the current code ref, active objective, existing checkpoint, relevant DB/asset witnesses, strongest test and review evidence, remaining blockers, and next route in the appropriate canonical surfaces. Keep private artifacts and credentials out of public repository docs. A genuine release or acceptance requires its own gate even when a tactical work unit finishes.

The CLIENT should be able to ask "what can I use today?" and get an answer about delivered capabilities and material risks. A newly opened context demonstrates durable continuity by recovering that answer from the repository and its authorized runtime evidence, without replaying the entire conversation.

Next: Chapter 04, Requesting Changes.

Sources and applicability

- Execution-team operating protocol
- Capability lifecycle and evidence
- Reference runtime
- Repository efficiency and disk hygiene
- Outcome-first work orders
- Goal/Plan source
- Host dispatch source

- Reference campaign controller

These operational practices describe contracts and examples; their presence in a repository does not certify any particular external host or application runtime.

04 — Requesting Changes

> Product decisions belong to the CLIENT. The Planner converts them into confirmed requirements, technical boundaries, delivery expectations and verifiable gates. I3 may revise its tactical plan within an authorized charter, but cannot unilaterally change its strategic Goal, forbidden effects, or the CLIENT's product definition.

In this chapter

1. Start with the outcome, not the code
2. A compact change-request brief
3. Which questions belong to you?
4. Change your mind without destroying history
5. Ask for quality without prescribing a stack
6. Prioritize instead of opening ten Goals
7. When the Planner needs a real decision
8. Five well-formulated requirements
9. Product change versus implementation defect
10. What to say when it works but is not useful
11. A short priority-setting script

1. Start with the outcome, not the code

The CLIENT describes who uses a function, what they want to do, what result they can observe, under what conditions it should fail, and what information must survive. The Planner translates that intent into confirmed requirements, dependencies, implementation boundaries, and acceptance gates. Technical preferences are legitimate, but routine decisions about APIs, database schemas, code organization and regression tests normally belong to engineering.

"Add a cancel button" is often incomplete. If cancellation affects remaining places, money, membership status, or history, simply drawing a button is not a complete solution. A useful question is: When an enrollment is cancelled, does a place become available again? What stays in the historical record? The answer defines observable behavior and the tests needed for acceptance.

The framework helps separate the product problem from a proposed implementation. An assistant may propose a button, an action menu or another interface, but the CLIENT's actual requirement is the correctly understood cancellation behavior. A visual mockup without the state and error behavior can help discussion; it is not by itself evidence of a finished capability.

2. A compact change-request brief

A product request need not be a lengthy specification. For meaningful changes, describe the actor, starting situation, desired action, visible outcome, important exclu-

sions, and retained information. Example:

> As the club administrator, I want to deactivate a member without deleting earlier payments. The deactivated record should disappear from active-member search but remain available to authorized staff in historical views.

The Planner should still clarify whether deactivation can be reversed, whether duplicate identities are allowed, and which historical information must be retained. You should not need to design the database or write the tests yourself.

Here is a complete example for activity enrollment:

Field	Example
User	Club administration staff.
Starting situation	A member wants to join an activity.
Visible outcome	The enrollment is saved only when a place is available and current rules permit it.
Exclusions	No online payments or automatic notifications in this version.
Retained information	Member identity, activity, date, status and subsequent changes.
Acceptance	Staff can enroll the member, see places remaining and cancel without erasing history.

Do not make the CLIENT fill out ten fields for every small correction. The purpose of the brief is to remove ambiguity that materially affects behavior. The Planner should make confirmed material changes durable; a conversation alone is not a reliable record for a new executor.

3. Which questions belong to you?

A question belongs to the CLIENT when it changes the product: Does an overdue payment block activity enrollment? Can an authorized user recover a deactivated record? Does the application need to work offline? May the system send emails, accept online payments, incur costs or publish member information? Those are decisions about people, risk and intended use.

Engineering questions include SQL schema details, API design, test frameworks, branch names, logging structure, internal retry algorithms and migration implementation. The Planner may explain consequences when a technical decision materially changes cost, security, portability or user-visible behavior, but should not ask the CLIENT to arbitrate a failed migration simply because an executor was unable to debug it.

When there are legitimate product alternatives, present them in understandable terms. For example: With a waiting list, staff must confirm a place when one becomes available; without a waiting list, the enrollment is rejected immediately. Explain cost and usability consequences, make a recommendation if appropriate, and ask for the actual product choice. There is no need to teach the CLIENT a database schema to obtain an answer.

4. Change your mind without destroying history

Product requirements can change. The system needs to know **which** decision changed, **what** replaces it, and **when** the new rule applies. For example:

> Previously members with overdue dues could not enroll in activities. From now on they may enroll, but the administrative view must clearly display outstanding debt. Record the previous rule as superseded and assess the effect on current screens, data and tests.

This may affect eligibility checks, the UI, persistent data, tests and ongoing campaigns. The Planner traces those dependencies and updates the canonical current requirement. A legitimate new requirement does not mean the earlier requirement was fictitious; it means its authority has changed. Two independent `CURRENT` documents silently demanding opposite behavior are a governance defect.

Similarly, the CLIENT may decide:

> I no longer want the application to permanently delete members from its interface. Deactivation must retain their payments and enrollments. Treat the old deletion rule as superseded, examine what happens to existing data, and explain the resulting user journey.

The engineering team owns the migration strategy, recovery, tests and interface changes. The CLIENT approves the new intended behavior and any reserved risk decisions. The current requirement must be reconstructible by a future session without replaying this chat.

If the change fundamentally alters the product boundary—for example, a local one-club tool becomes a multitenant Internet service—the Planner may need to invalidate or supersede a charter and reconsider architecture and dates. If the desired behavior was already in the charter and a test merely exposed defective code, I3 corrects the defect in the same campaign.

5. Ask for quality without prescribing a stack

“Make it professional” expresses a direction but is not a measurable acceptance condition. Describe what frustrates or misleads a real user: being unable to find a member, losing entered data on an error, not knowing whether a payment was saved, receiving a cryptic validation message, or triggering an irreversible action without confirmation.

A concrete request might be:

> The dues screen must show status, amount and the date of the latest payment without opening several unrelated windows. If saving fails, it must not display success. I prefer a clear, consistent interface over decorative complexity.

This can lead to usability criteria, integration checks and perceptual review without requiring a particular frontend library. A screenshot of the actual application with a precise criticism can be more useful than a list of adjectives. The CLIENT’s subjective judgment can be a real human/perceptual gate where product quality depends on the experience. No screenshot classifier or successful unit test should silently impersonate that acceptance.

Quality also includes negative behavior. A payment screen that works in the happy

path but duplicates transactions after a double click may be unacceptable. A form that technically saves but loses entered data after a validation error may fail a usability requirement. The Planner should map such outcomes to appropriate gates, while I3 decides how to implement and test the fixes.

6. Prioritize instead of opening ten Goals

Priority determines which capability should become usable first, within dependencies and resource constraints. For the club, the sequence might be: reliable member records, then dues, then activities and enrollments. If the third step requires the first two, document the dependency. Do not authorize ten competing capabilities merely because an assistant can generate ten plans.

Vague request	More useful direction
“Build the whole club system.”	“Make member management verifiable first; add dues after its data flow is dependable.”
“Fix whatever is wrong.”	“Cancelled enrollments are not releasing places; correct that behavior without losing history and demonstrate the complete flow.”
“Make it more modern.”	“Reduce the steps needed to find a member and check overdue dues; prioritize readability.”
“Add everything customers might want.”	“Validate the first administrator journey with real examples before expanding scope.”

If priorities shift while I3 is executing, the system should leave a recoverable checkpoint or properly close the active boundary before changing course. An abrupt priority change is not permission to abandon uncommitted valuable work, erase a campaign record, or silently redirect a Goal. The Planner accounts for dependencies, interference and the critical path.

7. When the Planner needs a real decision

The Planner should frame the question as a user consequence, describe the viable alternatives, identify any cost or risk differences, and state what the CLIENT alone must choose. A question such as “Should staff be able to reinstate a cancelled enrollment?” is useful. “Should I use an index with this SQL migration?” generally is not.

When you ask for status, a good answer has three parts: which behavior is physically proven now; what remains before the current objective can be accepted; and whether any indispensable product choice is waiting for you. It should not bury the answer under commit counts, token consumption, worker names, or a technical timeline with no visible result.

If you do not know the answer to a material product question, say so. Mark it open and continue safe work that does not depend on it. Engineering must not silently convert an unconfirmed preference into an authorized requirement.

8. Five well-formulated requirements

Search

> I want to find a member by name or membership number and distinguish active from inactive records. When there is no match, show a clear explanation. Do not expose personal data to unauthorized users.

This defines behavior, an empty-state experience, and a privacy requirement without choosing a search engine. The team can test authorized and unauthorized cases.

Edit

> Staff need to correct a member's phone number. Before saving, the screen must clearly identify which member is being edited. Invalid input must not erase the rest of the form; show an understandable error.

This describes navigation, state retention and feedback. A successful API endpoint test alone would not demonstrate the whole interaction.

Deactivation

> Deactivating a member removes the record from the active list, preserves payment history and allows authorized staff to inspect that history.

The word "deactivate" now has a testable meaning. If legal retention or deletion requirements later apply, treat those as a distinct decision and risk review.

Performance

> Search should remain usable at the number of members we expect in the first year. If the Planner needs that number to size a solution, ask me; do not assume an enterprise scale without evidence.

This connects performance to real workload instead of prescribing a premature database or caching technology.

New capability

> Add activity enrollments only after members and dues are usable. First confirm capacity, cancellation, overdue-payment and permission rules, then plan integration.

This prevents an apparently finished component from depending on incomplete or contradictory upstream capabilities.

9. Product change versus implementation defect

If the confirmed rule prohibits overdue members from enrolling but the application allows it, the implementation violates the current rule. I3 fixes and tests that defect inside the existing authorized scope. If nobody has decided how partial payments

affect eligibility, the Planner needs a product clarification before choosing a behavior. If the CLIENT now wants family members to pay for each other, that is a new product rule with potentially new identity and permission implications.

The distinction protects two responsibilities. Engineering should not demand CLIENT approval for every known bug. Equally, it should not convert a new user preference into an unreviewed technical assumption. Material changes update current requirements and preserve what they superseded, so a newly assigned agent can determine the valid rule.

A tactical-plan edit is not automatically a change to product intent. The protected strategic Goal may contain `goal_id`, `campaign_id`, `charter_ref`, `authority_refs`, `definition_of_done`, `hard_gates`, `forbidden_effects`, and `write_envelope`. I3 can revise steps, hypotheses and internal testing plans within that envelope. It cannot simply rewrite a forbidden external effect into an allowed one.

10. What to say when it works but is not useful

You can reject a capability because the real user journey is unusable even if saving technically succeeds. For example:

> The form correctly saves the member, but on the phone the confirmation button is outside the visible screen. I cannot accept the user journey until I can complete it on that device.

Attach a screenshot or describe the exact screen size, state, and expected behavior when possible. That is useful perceptual evidence. However, if mobile support was explicitly excluded from the current requirement, adding it may be a product scope change rather than an ordinary in-charter regression. The Planner should make that boundary explicit and, if needed, revise dates and dependencies. If mobile use was already required, I3 repairs the candidate and updates affected evidence within the same campaign.

The CLIENT's judgment is not a substitute for technical testing either. A beautiful screen can still have broken persistence, authorization checks or recovery. A genuinely accepted capability needs all the gates relevant to its intended use, each with its own owner and evidence.

11. A short priority-setting script

You can tell the Planner:

> My current priority is [visible outcome]. The user is [who]. They must be able to [complete journey] and must not [important prohibition]. My target date is [date], and the spending limit is [budget, if any]. Before implementation, tell me which product decisions remain open and which current capabilities depend on this change. Engineering owns the technology, tests, integration and in-scope recovery.

You do not have to repeat the wording literally. The objective is to make your intent recoverable and testable without converting every casual suggestion into an irreversible commitment. A capable Planner knows when to ask, when to verify physical state, and when to let I3 execute.

Next: Chapter 05 for errors, blockers and sensitive authorizations; Chapter 06 for capability acceptance.

Sources and applicability

- CLIENT decision boundary
- Interaction ledger
- Product discovery session
- Product requirements and technical baseline
- Capability lifecycle and evidence

The club examples demonstrate good product requests. They do not constitute an actual CLIENT decision for an adopter’s application.

05 — Errors, Incidents and Recovery

ArquitecturaRepo · Complete User Manual · v1.4 edition · English

> The objective of recovery is to restore a **truthful, verifiable state**, not merely to make an error message disappear. Preserve source, ownership, transaction journals, DB witnesses and relevant evidence. An ordinary in-charter bug is repaired by I3; a missing authorization or contradictory canonical currentness is a different class of blocker.

In this chapter

1. Preserve evidence first
2. Environment and account failures
3. When the control layer fails validation
4. Git is not an “undo problems” button
5. Repeated failures require a causal change
6. Security, privacy, and third-party rights
7. Exit an incident with evidence
8. Three complete recovery examples
9. Decision table for sensitive operations
10. What successful validation does not prove
11. The starter’s privacy and feedback controls

1. Preserve evidence first

Record the exact error message, the command or action that produced it, the relevant time, the affected repository and working tree, and the source or artifact reference if known. Do not instinctively delete a directory, reinstall the starter, discard local changes, or create another clone. Such actions can destroy precisely the information needed to recover safely.

The technical owner must classify the problem before choosing a repair route. Possible causes include an unavailable executable, missing permission, application defect, invalid governance contract, conflicting source ref, broken DB transaction, external provider failure, or reserved authorization. They are different incidents with different safe responses.

Observation	Safe first response
An integration test fails for an active feature.	I3 keeps the reproduction and repairs it inside the existing charter.
A control-layer validator rejects canonical state.	HOLD dependent actions; locate the conflicting relation and its owner.
Local and remote Git changes conflict.	Preserve both, inspect the exact delta and reconcile without guessing.
Real user data might be corrupted.	Stop sensitive writes, preserve backups and WAL/journal evidence, assess restoration.

Observation	Safe first response
Publishing or a paid action lacks explicit authority.	Do not perform the external effect while authorization is absent.
The coding host may have dispatched an external side effect before crashing.	Mark the effect unknown and perform readback before considering a retry.

The aim is not to reduce the number of red indicators by hiding them. The repair must address the cause and demonstrate the relevant gate on the current candidate. Read-only investigation and preservation normally remain possible while material changes are on HOLD.

2. Environment and account failures

Python or another command is missing. Verify the current directory, interpreter name, installed version and active environment. Use the declared project dependencies and trusted sources. Do not alter Product State to compensate for a missing executable. A coding assistant with access to the local environment can inspect the path; otherwise preserve the full command output for the person who can.

GitHub does not appear in your planning interface. Separate authentication, connector installation, permission scope, repository visibility and branch availability. A failed read does not justify writing files, importing more secrets, or granting access to an entire account. Confirm access to the exact authorized repository using a read-only request.

The coding agent opens the wrong project. Verify that the local folder exists and contains the actual product. Do not substitute the ArquitecturaRepo starter for `club-app` simply because both directories are named in the manual. Confirm write permissions and active worktree before making changes.

A session is interrupted or its usage limit is reached. The provider session is not the campaign. Recover the authorized Goal, active owner, last checkpoint, integrated candidate and pending effects from durable evidence. A time limit does not authorize a new campaign or a competing controller. If the host cannot resume the thread, the repository remains the source of continuity.

3. When the control layer fails validation

The installed `tools/validate_project.py` examines contracts and references in `.agentic-sdlc/`. It may reject a missing authority record, duplicate owner, inconsistent candidate/review relation or malformed current-state reference. It does **not** by itself test whether the application can connect to its DB, that a staff member can log in, or that payment history survives a restart.

Before editing anything, preserve the validator's exact output, the control-layer root, current Git ref, dirty files and relevant DB witness. The appropriate owner should determine whether the cause is a missing dependency, a contract mismatch, a genuinely stale reference, a partial transaction or a conflicting external edit.

Never force `READY` into a JSON file or erase a journal simply to make validation pass. If two actors plausibly own the same active effect, or a candidate has changed after an independent review, the correct response is to reconstruct authority and lineage.

A cosmetic field edit would make the report more reassuring while making the underlying state less truthful.

Transactional control-layer changes may update multiple related files. The framework documents explicit transaction recovery using before/after hashes and ownership. A pending journal can appropriately block further mutation. An external conflicting edit must be surfaced as a conflict, not silently overwritten to complete a stale transaction. The team should only resume when the relevant records agree and the recovery operation has been physically read back.

With SQLite, the main `.db` file alone may not represent the live data state while Write-Ahead Logging is in use. The associated WAL and transactionally consistent snapshot matter. A checksum of just the main file is therefore not automatically proof of current DB contents. The existing-governance attachment route preserves this distinction; it does not automatically repair a production database.

4. Git is not an “undo problems” button

When GitHub Desktop or the CLI reports a conflict, do not blindly choose “mine” or “theirs.” Both sides may contain legitimate work. First inspect the exact base, local modifications, remote delta and current ownership. A trusted checkout should not be deleted merely because a diff is inconvenient.

The starter includes `tools/repo_delta.py` for bounded incremental inspection and `tools/workspace_gc.py` for controlled cleanup. Neither is permission to delete source, accepted artifacts, unknown human files or current evidence. Begin cleanup with a read-only inventory and dry run; deletion needs positively established ownership and disposability. A directory that appears old is not necessarily garbage.

Use one persistent checkout per repository when safe, and temporary shared-object worktrees only for genuine concurrency. Do not accumulate cloned repositories, large render caches or duplicate evidence “just in case.” At the same time, do not make disk hygiene a pretext for destroying lineage, backups, active transaction records or artifacts needed to falsify or accept a capability.

`git reset --hard`, forced replacement of a branch and recursive deletion are not general recovery recipes. When a rollback is truly necessary, identify the precise target, restore point and affected data first. An application rollback may need a corresponding schema or data strategy; moving Git alone cannot unwind a production DB transaction.

5. Repeated failures require a causal change

Running the same command with the same inputs and assumptions repeatedly is not progress. When a class of failure recurs, I3 consolidates its observations, inspects the affected subsystem and interfaces, strengthens relevant regression and integration tests, and obtains independent review. It remains the same authorized campaign until there is genuine strategic invalidation.

For costly work, the bounded work-order contract establishes budgets, retry limits and stop conditions. If successive attempts produce no delivered outcome, I3 changes the method or reports the exact causal blocker. The planner should not generate an endless sequence of new campaigns simply to relabel the same failure. Likewise, passing a narrow unit test after a patch is not enough if the actual user workflow remains broken.

The useful terminal is one of: physically delivered capability with relevant checks, an explicitly bounded blocker with evidence and owner, or a recoverable checkpoint with a known legal next action. A stream of commits, model calls or advisory documents does not substitute for any of these.

6. Security, privacy, and third-party rights

A technical charter does not imply permission to expose private information, pay an API, disclose credentials, remove protected records or publish another person's material. These operations have their own reserved authority and risk boundaries. Use least privilege, synthetic data in unauthorized demos, authenticated secrets mechanisms, lawful data handling and source-appropriate license checks.

- Never place passwords, tokens, private keys or connection strings in prompts, issues, repository history or public build logs.
- Work with synthetic or appropriately deidentified information when a review or demonstration does not need real people.
- Before a destructive migration or cleanup, identify affected records and verify an appropriate backup and restoration route.
- Before publishing third-party images, code, music or video, establish the rights required for the actual use. The starter's MIT license covers its own licensed material, not unrelated assets.
- Before billable provider calls, obtain the authorized budget, rate/cost bounds and meaningful stop conditions. A connection failure does not justify unlimited paid retries.
- Before exposing an application publicly, verify authentication, authorization, network configuration, data paths, secrets and the explicit decision to deploy.

Privacy, intellectual property, security and operational safety are related but distinct gates. A human's acceptance of a sample's visual quality does not create a copyright license. A code test passing does not create permission to publish personal data. Keep each decision with its actual owner.

7. Exit an incident with evidence

An incident can be closed only when the recovery report explains what failed, its cause, what changed, what source/data were preserved or restored, what physical checks were performed, and which risks or gates remain open. The report names the current candidate and the owner of the next action. I3 can confirm a technical repair, while the Planner evaluates whether the capability's strategic conditions are again met.

If the current state cannot be reconstructed, or there is no trustworthy recovery route for critical data, say so plainly. An explicit HOLD with a precise missing witness is safer than pretending that the original application state survived. Continue safe investigation rather than authorizing dependent mutations against unknown conditions.

8. Three complete recovery examples

Incident A — The coding session ends during a campaign

The coding host disconnects while a member search feature is in progress. Do not send the same high-level request into a new isolated session and hope the resulting work is equivalent. Inspect the actual checkout for uncommitted writes, locate the campaign journal and latest checkpoint, identify the current owner/lease, and read any pending external effect receipts. Confirm what candidate was integrated and what remains outstanding.

If source and authority agree, resume the existing campaign at the next legal step. If a critical reference is absent, record which witness is missing and HOLD only work that depends on it. Provider-session limits neither create new product requirements nor invalidate the current charter.

Incident B — A multi-file state update is interrupted

A transition intended to update bootstrap, Product State, ownership and checkpoint stops halfway. Several files may now disagree about which version is current. The framework's state-transaction journal exists so the owner can distinguish fully applied changes from partial or conflicting ones. Inspect it before running further mutations.

Use the documented recovery command belonging to the tool that initiated the transition, such as `campaignctl recover` for a compatible reference-runtime transaction. The transaction mechanism advances only when expected before/after hashes still justify each write. If someone else edited the same canonical file, the correct outcome can be `CONFLICT`. Do not manually erase the journal, edit statuses or overwrite the unrelated change to force a clean report.

Incident C — Tests pass but the real application fails

A dues calculation passes unit tests, but the member screen shows an outdated amount after saving. Capture the exact source candidate, user steps, environment, expected and observed behavior, and relevant DB state. The unit gate passed; the end-to-end or persistence gate did not. Exercise the failing interaction against the current candidate and repair the shared state or integration path.

If the issue is within the existing charter, I3 owns the correction and updated independent review. Repeatedly rerunning the same unit test cannot validate a missing UI refresh or DB commit. The incident is resolved when the physical journey and adjacent affected checks demonstrate the repaired behavior.

9. Decision table for sensitive operations

Proposed action	Primary risk	Minimum pre-execution boundary
Show a demo using real member records.	Personal-data disclosure.	Use synthetic records or establish appropriate consent, purpose, permissions and protection.

Proposed action	Primary risk	Minimum pre-execution boundary
Create a GitHub token.	Excessive access or leakage.	Least privilege; keep the secret out of source and logs; revoke it if exposed.
Delete apparently temporary folders.	Loss of live work or evidence.	Confirm exact owner, disposable status and dry-run target.
Open a public deployment.	New attack surface, private-data exposure and cost.	CLIENT authorization, security/network preflight and observable rollback plan.
Activate a paid API.	Unbounded charges or retries.	Explicit budget, operation bounds, measurement and terminal failure behavior.
Publish third-party media.	Unauthorized reuse.	Verify source identity, license, applicable scope and attribution.

A general “you may work in this project directory” is not blanket authorization for irreversible, external or billable effects. Engineering chooses technical safeguards; the CLIENT makes product and reserved exposure decisions.

10. What successful validation does not prove

A passing `validate_project.py` result may show that structural control records are internally consistent. It cannot prove that a backend is reachable, a real DB survives restart, an endpoint enforces all required permissions, or a user can complete the intended journey. A synthetic adoption test does not authorize an untested production migration. An attractive screenshot does not replace real interaction testing.

When repairing an incident, rerun the gate that actually failed and nearby checks plausibly affected by the fix. Identify the new candidate and reviewer. Do not invalidate unrelated evidence for no reason, but do not promote a material code change using stale review results. If code, DB and current Product State still contradict each other, keep dependent changes on HOLD until physically reconciled.

The same separation applies to distribution: a passing source build is not by itself proof of a published archive’s contents, checksum or availability. Release readback is a distinct check on the exact distributed bytes.

11. The starter’s privacy and feedback controls

In v1.4 the scaffold’s local strict-metadata feedback report configuration is enabled by default. A periodic GitHub Actions scheduler can also be generated. External collection is not bundled in the public release; sending feedback externally requires the separately documented explicit opt-ins. Review the privacy feedback contract and the installed executable workflow before enabling any network effect.

The `--no-feedback-workflow` scaffold option avoids installing the periodic scheduler, but **does not disable the local report generator**. To turn off the latter, set `enabled=false` in the installed feedback configuration. Be explicit about the distinction

rather than describing either option as “disabling all telemetry” without checking its actual scope.

None of these defaults proves that a subsequently developed application handles its own personal data safely. Each adopting project must assess its actual credentials, logs, data flows, permissions, network interfaces and retention requirements.

Next: Chapter 06, Closing Capabilities and Acceptance Gates.

Sources and applicability

- Security, privacy, third-party rights and external actions
- Transaction recovery
- Privacy-preserving architecture feedback
- Existing-governance read-only preflight
- Reference transaction implementation
- Workspace cleanup implementation
- Repository delta inspection

These examples are hypothetical. Any real recovery or deletion requires direct evidence from the affected target and the appropriate authorization.

06 — Closing Capabilities and Acceptance

ArquitecturaRepo · Complete User Manual · v1.4 edition · English

> A capability closes against an **exact candidate and the applicable evidence**. An I3 terminal invites strategic review; it does not automatically approve a product, satisfy a human gate, grant publication rights, or deploy anything. This chapter separates those decisions and explains how to correct in-scope failures without restarting the architecture.

In this chapter

1. A capability is a product result
2. Five components of closure
3. The exact candidate is the review unit
4. Missing evidence does not require an artificial campaign
5. Accept, resume, or change objective
6. Periodic review and the stopping rule
7. An auditable closure record
8. Rejection example: correct without restarting
9. Usable product versus green framework
10. When to end a session or stop work

1. A capability is a product result

“Member management” is not finished when someone draws a form or commits its component. It is finished when the agreed behavior works in the relevant environment, required information survives, applicable checks pass, current state and lineage are coherent, and the appropriate strategic and human authorities have accepted the result.

The Planner defines the capability’s boundary and charter. I3 integrates an exact candidate, obtains independent review when required, and returns source-bound evidence. The Planner audits that terminal against current requirements and charter acceptance criteria. The CLIENT decides reserved product preferences, perceptual quality and external effects when applicable.

A cycle is not a fixed number of prompts or one coding session. A campaign can survive ordinary retries, a disconnected host and several execution checkpoints. Those changes do not create new authorization or erase earlier evidence. Conversely, a large number of commits and a confident report do not establish that the person using the application can complete the journey.

2. Five components of closure

Component	Closure question
Visible result	Can the intended person perform the requested task in the relevant environment?
Applicable checks	Have the unit, integration, DB/recovery, security, runtime, UI and other gates relevant to this capability been exercised?
Coherent current state	Is the candidate's exact code and configuration known, together with current requirements, Product State and affected data/artifacts?
No concealed material blocker	Does any unresolved defect invalidate the claimed result despite passing unrelated checks?
Competent acceptance	Have the Planner and, where applicable, the CLIENT inspected evidence within their authority instead of accepting a generic `DONE`?

Not every capability needs every possible test. A static internal tool may have no meaningful perceptual gate, while a member-payment application must address data persistence and the human-facing payment journey. The crucial rule is that one gate may not silently replace a different gate that actually applies. A valid JSON schema is not evidence of durable CRUD operations; a unit test is not proof that a person can find the function on screen.

Use the following gate vocabulary consistently:

- **Unit:** individual logic behaves correctly under controlled inputs.
- **Integration:** interacting components satisfy their interface contracts.
- **DB/currentness:** relevant records, migrations, source bindings, and runtime witnesses agree and persist as required.
- **Semantic/generalization:** the behavior matches the actual intended rule beyond one narrow fixture.
- **Physical:** the program or artifact genuinely runs, opens, renders or performs the journey on the relevant target.
- **Perceptual/human:** a qualified person assesses the experience or subjective product criteria where necessary.
- **Release:** the exact distributable candidate, permissions, rights and downstream publication conditions are satisfied.

The evidence, owner and need for each gate should be proportional to the feature. Adding a dozen unnecessary checks can reduce delivery without increasing safety; omitting a material gate can produce a convincing report about a broken product.

3. The exact candidate is the review unit

An independent reviewer must inspect the same source and state that will be proposed for acceptance. Depending on the capability, that may require code ref, configuration, DB snapshot or schema version, exact asset hashes and runtime environment. A review against candidate A cannot simply be transferred to B because both have the

same branch name or filename.

The normal sequence is: integrate candidate, freeze its relevant identity, perform independent review and required physical tests, then submit a terminal for strategic audit. If a producer materially changes the candidate after that review, the affected approval needs to be renewed for the new candidate. This does not require erasing unrelated stable evidence, but it does forbid treating a stale favorable review as an endorsement of changed bytes.

The review-freeze and terminal-integrity protocol defines this relationship. The optional reference controller can model freeze, review and terminal states; it cannot impersonate the actual product owner or automatically demonstrate a proprietary host's behavior.

Even after technical review passes, human acceptance, real deployment evidence, privacy checks, third-party rights or public-release authorization may remain open. Record those as separate downstream gates. A technical I3 terminal means "candidate ready for the appropriate review," not "every authority has approved every effect."

4. Missing evidence does not require an artificial campaign

If the remaining defect is within the authorized charter, the Planner sends I3 a **consolidated** list of falsifiers and their evidence. I3 researches, repairs, integrates, retests and arranges a new appropriate review in the same campaign. It should not ask for a new strategic plan after every failed test.

After repeated returns of the same defect class, the right response is broader subsystem hardening and independent review, not a chain of narrowly worded microfixes. A newly discovered material product choice, changed authorization or incompatible strategic interface may instead require the Planner to revisit the charter boundary. The distinction prevents both governance churn and uncontrolled scope expansion.

A useful terminal summary is short:

> **Candidate:** exact ref and environment. **Observed result:** the user journey actually exercised. **Passed gates:** linked evidence. **Open gates:** the specific limitation and impact. **Remaining risks and new product decisions:** separated. **Next action and owner:** one clear current route.

This is a readable view of canonical evidence, not a second Product State to maintain in parallel. References must resolve to the actual reviewed candidate, not merely a similarly named log or a prior version.

5. Accept, resume, or change objective

Accept. When the right authorities have the right evidence, the Planner records the accepted capability and updates canonical Product State and current pointers. The CLIENT completes any reserved perceptual or product acceptance. Only then should dependent downstream work rely on the newly accepted result.

Resume after interruption. A new executor or Planner reconstructs the current campaign and checkpoint. It does not mark started work complete because a service limit was reached, nor does it invent approval because a session ended. Current source, DB, owners and evidence remain stronger than remembered conversation history.

Change priorities or product intent. A temporary scheduling adjustment is not necessarily a requirement change. A changed rule about data or user behavior is. Record supersessions and make unfinished work recoverable. Do not keep two contradictory active charters for the same concern, and do not erase earlier valid work merely because the priority order changed.

6. Periodic review and the stopping rule

Once a week, the CLIENT should be able to ask: Which capabilities are accepted? What is the current priority? What can actually be demonstrated today? Which target date is still justified? What material risk remains? Is there a product decision only I can make?

The answer should be grounded in source and evidence, not a tally of commits, agents, hours, review comments or tokens. Such activity may be necessary, but it is not the delivered result. A framework can become counterproductive if it adds more reporting than it removes uncertainty.

The reusable starter has a stopping principle too: once release gates are met, do not continuously add generic policy and machinery merely to keep engineering active. New mechanisms need an identified class of failure or genuine capability gap that existing controls do not address. Apply the same discipline inside products. Both premature acceptance and infinite searching for immaterial microdefects undermine trustworthy closure.

7. An auditable closure record

A new worker who was absent during implementation should be able to understand the evidence without reading the entire chat. A compact closure record provides a map to canonical sources:

Field	What must be traceable
Candidate	Exact source/state/artifact identity reviewed.
Outcome	End-to-end journey, with concrete input, action and observable output.
Tests	Relevant execution outputs and environment.
Persistent state	Post-write readback, restart and recovery checks where applicable.
Security and rights	Proportional checks for sensitive data, network exposure or third-party material.
Independent review	Reviewer identity, scope and exact candidate examined.
Post-review changes	Whether anything material changed, and which checks were renewed.
Human gate	Authentic acceptance or rejection where real usability or subjective quality matters.

Field	What must be traceable
Next route	Accept, return to I3, request product decision, or HOLD with a specific cause.

Do not create another competing `CURRENT` file for the convenience of this table. The record is a way to navigate existing canonical evidence. If a link leads to an old ref, a missing artifact or a test fixture unrelated to the claimed product, the record is insufficient for acceptance.

The two broad families of framework release evidence should also remain separate: internal code/contract/transaction correctness, and a complete installed lifecycle on a real portable target. Neither family automatically certifies UX or business correctness of an unrelated application built with the framework.

8. Rejection example: correct without restarting

Suppose a club application now supports creating and deactivating members. Independent review discovers that deactivation erases payment history, contrary to the charter. That is an implementation defect within the already authorized capability. The Planner returns one clear falsifier: Deactivation of a member with payments must preserve historical transactions after a restart; currently payments disappear.

I3 examines persistence and relationships, fixes the cause, adds meaningful regression tests, demonstrates the record after restart, integrates the new candidate and obtains a new independent review. It does not open a separate “fix deactivation” campaign or ask the CLIENT whether a foreign key should cascade.

If the reviewer inspected version A, and the persistence change produced version B, the PASS for A cannot accept B. This is why exact candidate identity matters. Evidence from unaffected tests may remain reusable when their source dependencies are unchanged, but the affected persistence and review gates must be updated.

Now consider a different event: the CLIENT requests a new permission-based reactivation feature. That is not a repair of the known deletion defect; it is a new product rule. The Planner confirms the desired behavior, assesses existing interfaces and risk, and decides whether it fits the authorized envelope or requires strategic revision.

9. Usable product versus green framework

A campaign may pass governance validation and put its reference controller in a terminal state while producing no usable application screen. Product acceptance must demonstrate the actual requested result. For a local club app, open it, register a fictional member, find the record, edit it, deactivate it, and confirm that payment history survives. If persistence matters, restart the application and repeat the relevant readback. If permissions matter, test the unauthorized path too.

ArquitecturaRepo’s own release gate is different. `tools/verify_v1_release.py` checks a concrete snapshot of the reusable framework and synthetic fixtures. Source verification, Linux/Windows portable install checks, and release artifact readback are distinct checks. A green framework release does not certify every possible adopted installation or application made with it. An external/native host adapter requires its own demonstrated compatibility.

The v1.4 support matrix explicitly distinguishes what was exercised from what remains outside portable proof. Keep those limits visible when reporting a project's progress. The phrase "all tests passed" must say whose tests, on which candidate, and against which capability.

10. When to end a session or stop work

After strategic acceptance, persist the change to current Product State, maintain source-bound lineage and unblock only dependencies justified by that acceptance. After an in-scope rejection, return to the same campaign. After a material invalidation, revisit the strategic boundary with the appropriate owner. When evidence cannot establish currentness, leave a concrete HOLD rather than guessing.

The CLIENT can finish a day without understanding the internal controller, but should know which capabilities are usable now, which are not, what decision is required of them, and when the next milestone will be reviewed. A project that can answer those questions only while yesterday's chat remains open has not achieved durable continuity.

Next: Chapter 07, A Worked Club Example from Discovery to Acceptance.

Sources and applicability

- Review freeze and terminal integrity
- Capability lifecycle and evidence
- Executable validation
- Reusable-framework release gates
- v1.4 supported installation and distribution scope
- Release verifier source

Acceptance is always source- and context-specific. These examples do not create authority or substitute for actual evidence in an adopting project.

07 — Worked Example: A Club Management Application

ArquitecturaRepo · Complete User Manual · v1.4 edition · English

> This is a **fictional teaching example**, not a deployed application, a generated backend, or proof that an AI model can automatically deliver an accepted CRUD system. The examples, records and candidate names below are illustrative. On a real project, requirements, source, DB/WAL state, tools and acceptance evidence must come from that project.

In this chapter

1. The example product: a club with concrete needs
2. Ask questions that change the product
3. Install without confusing a starter with an application
4. First capability: member management
5. From Planner to execution: a complete request
6. What to test before acceptance
7. Second capability: dues, without expanding scope
8. Evaluate the whole example
9. The first cycle with fictional data
10. Record a finding and route it correctly
11. The second cycle depends on the first
12. Final exercise: recover the project in a new context

1. The example product: a club with concrete needs

An association employs one administrative worker who registers members, tracks dues, manages activities and records enrollments. The application must preserve relevant history, avoid oversubscribing activities and show who is up to date with dues. The first release is simple, internally operated, and deliberately excludes online payments, a dedicated mobile app, fiscal invoicing and external integrations unless a later product decision justifies them.

That product description gives a direction, not an implementation. It does not decide whether the source will be Python, Java or another language; it does not define a verified schema or host; and it does not authorize anyone to access genuine membership data. The real codebase, stack, threat model, operating environment, and exact acceptance gates must be reconstructed when an actual project exists.

A useful capability sequence is member identity → dues and payment history → activities and capacity → enrollment eligibility and cancellation. This is a dependency relationship, not an invitation to create four agent hierarchies or four independent products. If the first capability lacks trustworthy member identity, subsequent payments cannot safely refer to those members. If the second capability cannot produce a current dues status, enrollment eligibility cannot truthfully rely on that status.

2. Ask questions that change the product

The Planner begins by asking the CLIENT who operates the club, whether the program is restricted to one computer or accessed over a network, what personal data is held, how backups should work, the budget, target date and which actions may alter historical records. A plausible first rule is: An administrator may deactivate a member, but earlier payments must remain available to authorized staff.

Other decisions must be explicit. Does any overdue dues period block a new enrollment? Can the member enroll when they have a payment arrangement? Does cancellation free a place immediately? Is there a waiting list when an activity is full? Do different employees need different rights? Without confirmed answers, a Planner should not turn guesses into the current product specification.

An illustrative first-version baseline is:

Area	Requested first-release behavior
Members	Create, edit, search and deactivate, while preserving historical relations.
Dues	Register payments, identify overdue periods and inspect history.
Activities	Name, schedule, capacity and available places.
Enrollments	Register/cancel participation with capacity and dues rules.
Exclusions	No online payment, mobile application, mass notifications or complex dashboards.

Every line is provisional until the CLIENT confirms it. Dues and enrollments imply persistent application state, so the Planner would evaluate the `STATEFUL` profile and the data, idempotency, recovery and security controls justified by real usage.

3. Install without confusing a starter with an application

For a new ungoverned club repository, the installer may create the control layer with the justified profile. From a v1.4 source checkout in PowerShell:

```
& "python" "tools\scaffold_project.py" --target "..\club-app" --profile "STATEFUL" --p
```

The result is `.agentic-sdlc/` inside `club-app/`, initially in `HOLD / UNTRUSTED_CONTEXT`. The installer does not claim that any existing club application runs, that its DB is configured, or that a development campaign has authority. If `club-app/` already has canonical governance, use the read-only governed-project attachment; do not install a second control layer or force over the first.

After creating an initial scaffold, the installer provides read-only status, inventory and reconstruction commands for that target. Chapter 01 includes the v1.4 sequence. The Planner and I3 then inspect source, inventory, baseline, requirements and existing state. They prepare a reviewed adoption specification, run preflight, promote coherent records and demonstrate fresh-context reconstruction.

The CLIENT reviews the actual scope and approves product behavior. Engineering selects a justified stack, DB, network exposure, permissions, backup strategy and

dated delivery milestones. Only when the relevant baseline and currentness gates are coherent should the Planner authorize a first material charter.

4. First capability: member management

Starting with members is a product dependency choice. Payments, activity enrollments and other records need a stable member identity. The first campaign's outcome might be:

> An authorized administrator can create a member, inspect the record, correct details, find it through search and deactivate it. Existing historical payments or enrollments remain intact and available through the permitted interface.

The charter should name affected interfaces, data ownership, expected error behavior and applicable gates. Tests include duplicate identities, invalid data, actual writes and subsequent reads, persistence after application restart, correct active/inactive filtering, safe historical relations, and a complete user-interface journey when the product has a UI. Independent review must inspect the exact integrated candidate.

The CLIENT does not need to specify the ORM, transaction boundaries, table names or folder layout. Engineering should explain any choice that changes cost, accessibility or reserved risk. Existing data must be treated as current evidence, with migrations and backups handled through an authorized process. No sample record from this manual should be confused with a real user.

5. From Planner to execution: a complete request

A dispatch can be compact while still giving I3 what it needs:

> **Goal:** Deliver the member-management capability in the current authorized charter.
>
> **Context:** Reconstruct confirmed requirements, canonical Product State, real repository and exact source ref. Identify affected member records and current DB state.
>
> **Constraints:** Preserve history; do not introduce dues processing, notifications or other unrelated features; remain within the confirmed network and spending boundaries.
>
> **Execution gates:** Physically exercise create, search, edit, deactivate and durable readback. Test invalid and duplicate inputs, plus relevant authorization and concurrency paths. Independently review the integrated candidate.
>
> **Done when:** Return exact candidate ref, verifiable physical evidence, applicable test results, review findings, remaining gates and an honest strategic terminal.

I3 may break the work into tactical steps, implement and review in bounded units, debug and retry. If a double click creates the same member twice contrary to the confirmed uniqueness rule, I3 fixes the defect in the same campaign; there is no need to open a second strategic campaign for duplicate prevention.

Where work has multiple material operations or significant cost, add a bounded work order with a concrete inventory, method, budgets and stop conditions. A work order

is part of the existing authorized campaign; it cannot expand the CLIENT's product request or call a paid provider without authorization.

6. What to test before acceptance

Use fictional records such as `MEMBER-001`. The test plan below is a specification of actions to perform, **not evidence that any such application exists**:

Step	User action	Expected observable outcome
1	Open the application as an authorized administrator.	The real login or local-access mechanism permits the intended role.
2	Create `MEMBER-001` with valid synthetic details.	Exactly one member record is saved; success and identity are visible.
3	Search by an allowed identifier.	The correct record appears, without revealing unauthorized data.
4	Change a fictional phone number.	The new value appears after saving and a fresh read.
5	Close and reopen the application.	The change persists through the actual DB/recovery path.
6	Deactivate the member.	Active search no longer returns the record as active.
7	Inspect history with appropriate permissions.	Historical payment and enrollment records remain available.
8	Attempt an invalid/duplicate operation.	The app rejects it clearly without silently corrupting state.

For each step record the exact candidate, test input, expected result, observed result and environment. A lack of visible errors is not proof of persistence; inspect the post-write state through the actual application or an appropriate data readback. If two employees can act simultaneously, exercise the concurrency scenario in that environment rather than inferring it from a sequential test.

Do not put real members' personal information in screenshots, public issue descriptions or videos. When the UX matters, a qualified human should complete the journey and provide the perceptual result, separate from automated tests. When the data matters, check current DB/WAL witnesses, relevant recovery behavior and rollback protections in the real environment.

7. Second capability: dues, without expanding scope

Once members are strategically accepted, the CLIENT may request recording dues payments, viewing who is current and retaining a complete payment history. The

Planner confirms billing periods, partial payments, overdue states, corrections, permissions and who may edit financial records. Engineering then chooses appropriate persistence, idempotency, audit and recovery mechanisms.

For example, the CLIENT might decide: If a payment was recorded in error, administrators reverse it with an audit entry instead of permanently deleting the original transaction. This is a product decision with technical consequences. The Planner records it; I3 implements the schema, behavior and tests. The implementation should reject accidental duplicate effects when a user repeats an action or a network request arrives twice, according to the confirmed rule.

Activity enrollment and capacity management should follow when their upstream member identities and dues status are trustworthy. Avoid adding microservices, a cloud payment processor, a complex permission system or a mobile app merely because those technologies are available. Each additional mechanism needs a real product or risk justification.

8. Evaluate the whole example

At the end of this exercise, you should be able to explain what the CLIENT decided, how the Planner confirmed and persisted requirements, what physical evidence I3 produced, why a green test cannot automatically establish acceptance, how the team handles an ordinary in-scope defect, and how a fresh worker reconstructs the current state.

The objective is not to memorize every internal file name. It is to know where each decision belongs and to require an observable result whose provenance remains accessible after the chat ends. A system with elaborate role descriptions but no demonstrable member journey has not delivered the product outcome.

9. The first cycle with fictional data

To practice, invent a member named “Test Member A” and a fictional contact number. Confirm which fields the product genuinely requires and which identifier must be unique. The Planner expresses the requirement, while engineering chooses the storage model, uniqueness controls and transaction behavior.

Perform the full path: create the member, see them in the list, search, edit, restart the program, read the edited record, deactivate, and inspect permitted history. Record expected and observed outputs at every point. The initial UI screenshot is just the beginning of this proof; it does not demonstrate what the DB actually persisted.

Add negative cases: duplicate unique identifiers, invalid fields, simulated controlled DB disconnection during save, and repeated submissions that might reach the backend twice. The exact expected error depends on the confirmed product rule; silently inconsistent data is not an acceptable default. Where several operators are in scope, verify interleaving or simultaneous actions rather than claiming concurrency safety from a single-user fixture.

10. Record a finding and route it correctly

Suppose the CLIENT tries deactivating “Test Member A” and sees that payment history disappears. If retention was already a confirmed requirement, the behavior is an

implementation defect. Record the candidate ref, steps, observed output and affected gate. The Planner sends that consolidated falsifier to I3. The controller repairs the persistence path, reruns physical and data checks, integrates a new candidate and obtains an appropriate independent review.

Now suppose the CLIENT asks whether an inactive member may be reactivated with the same member number. If no earlier decision defines that behavior, do not label either possible outcome a bug. The Planner asks the product question, records the new requirement and evaluates its impact on current authority and interfaces. A new rule and a violation of an existing rule are not equivalent events.

The team should not manufacture acceptance on the basis of the manual's fictional narrative. If no actual application has been built, the correct status of all physical demo steps is **NOT RUN**, not PASS. The point of the exercise is learning which evidence would be necessary.

11. The second cycle depends on the first

Dues need a stable member key and clear rules about partial payments, duplicates, deadlines and corrections. The CLIENT confirms these rules; the Planner records them against the accepted member capability. I3 then implements them with sufficient integration, DB and recovery checks. The next campaign should use the accepted candidate and its current interfaces, not a remembered earlier implementation.

Before implementing activity enrollment, verify that the dues status consumed by eligibility rules actually exists and is current. Starting with enrollments while dues policy remains provisional can lead to duplicated or contradictory checks. This is a critical-path relationship between product capabilities, not a requirement to run more agents or create a complex organization.

12. Final exercise: recover the project in a new context

Imagine the work stops and a different Planner enters tomorrow. Without giving that worker the previous chat, the repository and authorized runtime should reveal whether member management is accepted, which exact candidate and evidence support it, which dues questions remain open, who owns execution, and what next action is permitted.

If two campaigns claim to be active for the same resource, or an old report is labeled current, the worker should recognize and reconcile that contradiction instead of guessing. If a source or DB witness is missing, it should declare the specific uncertainty and HOLD dependent work, while continuing permissible read-only investigation.

The exercise is complete when you can distinguish: a genuine product decision, an in-scope defect, a temporary environment failure, a required external authorization and an actual strategic charter invalidation. Demonstrable physical results and independently reconstructible current state—not installation alone—are the final learning outcome.

Sources and applicability

- Minimal synthetic example

- Fork and bootstrap
- Product discovery and dated delivery
- CLIENT boundary
- Campaign charter standard
- v1.4 public start
- Existing governed-project preflight

The source-backed instructions and fictional club examples are educational. None of them establishes a real application, permission, accepted feature or tested deployment.